

Accepted Manuscript

A new efficient approach for mining uncertain frequent patterns using minimum data structure without false positives

Gangin Lee, Unil Yun

PII: S0167-739X(16)30303-X

DOI: <http://dx.doi.org/10.1016/j.future.2016.09.007>

Reference: FUTURE 3156

To appear in: *Future Generation Computer Systems*

Received date: 25 March 2016

Revised date: 9 September 2016

Accepted date: 10 September 2016

Please cite this article as: G. Lee, U. Yun, A new efficient approach for mining uncertain frequent patterns using minimum data structure without false positives, *Future Generation Computer Systems* (2016), <http://dx.doi.org/10.1016/j.future.2016.09.007>

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.



A New Efficient Approach for Mining Uncertain Frequent Patterns using Minimum Data Structure without False Positives

Gangin Lee and Unil Yun*

Department of Computer Engineering, Sejong University, Seoul, Republic of Korea

E-mail: ganginlee@sju.ac.kr, yunei@sejong.ac.kr

Abstract

The concept of uncertain pattern mining was recently proposed to fulfill the demand for processing databases with uncertain data, and various relevant methods have been devised. However, previous approaches have the following limitations. State-of-the-art methods based on tree structure can cause fatal problems in terms of runtime and memory usage according to the characteristics of uncertain databases and threshold settings because their own tree data structures can become excessively large and complicated in their mining processes. Various approximation approaches have been suggested in order to overcome such problems; however, they are methods that increase their own mining performance at the cost of accuracy of the mining results. In order to solve the problems, we propose an exact, efficient algorithm for mining uncertain frequent patterns based on novel data structures and mining techniques, which can also guarantee the correctness of the mining results without any false positives. The newly proposed list-based data structures and pruning techniques allow a complete set of uncertain frequent patterns to be mined more efficiently without pattern losses. We also demonstrate that the proposed algorithm outperforms previous state-of-the-art approaches in both theoretical and empirical aspects. Especially, we provide analytical results of performance evaluation for various types of datasets to show efficiency of runtime, memory usage, and scalability in our method.

Keywords

Correctness, Data mining, Existential probability, Frequent pattern mining, Uncertain pattern

1. Introduction

With the development of networks and IT devices, large volumes of data have been generated in various

* Corresponding author. Email: yunei@sejong.ac.kr

application fields. As more and more data have been generated and accumulated, various methods for data analysis and management have been proposed, and researchers in various areas have developed techniques for dealing with such data including privacy-preserving [13, 23, 24] and cloud-based [27, 28, 29, 33, 34] techniques. Meanwhile, as approaches for finding useful knowledge or information hidden in such large-scale databases, data mining [3, 21, 26, 42] have been utilized in various application fields such as analyzing biomedical data [10, 36], traffic data analysis [9], network data [37], and mobile data [4, 43]. Frequent pattern mining [2, 12, 30, 32] is one of the most interesting areas in data mining. Due to its extensive usefulness, various relevant works have been studied such as mining high utility patterns [35, 47, 51], representative patterns [45, 49], erasable patterns [14, 46], stream patterns [5, 39], privacy preserving patterns [50], sequential patterns [5, 36, 49, 54], and graph patterns [15, 48]. Frequent pattern mining extracts meaningful pattern information from various types of databases, and then users employ the mining results for data analysis.

Traditional frequent pattern mining focuses on databases with exact information [2, 11, 30]. However, in contrast to such precise data that can express whether each item within the data is present or absent, there are uncertain data that cannot exactly denote states of items as presence or absence according to the environments and characteristics of accumulated data. That is, various applications in the real world may have not only precise or certain data but also a variety of uncertain data such as sensor data [52, 54], personal identification data [53], and spatiotemporal query data of moving objects [6, 44]. Various approaches have been devised to deal with such uncertain data [1, 7, 8, 25], but they still have limitations in the following aspects. State-of-the-art uncertain pattern mining algorithms [17, 18, 22, 40] extract uncertain frequent patterns on the basis of their own tree structures and mining techniques. In this process, they load given uncertain data into the main memory in their own tree forms. Therefore, the sizes of tree structures can become excessively large depending on the sizes of inputted uncertain databases and threshold settings. In the worst case, tree sizes may become too large to load their data into the main memory; moreover, performing the mining operations from such huge tree structures can also cause fatal computational overheads. In order to overcome the drawbacks of uncertain pattern mining, various approximation methods have been proposed [19, 20, 38, 41]. However, since they improve the mining performance at the cost of the accuracy of their mining results, they cannot mine a complete set of uncertain frequent patterns from given uncertain databases.

In various pattern mining areas, an enormous number of mining techniques and applications have been devised. However, most of them follow the framework of famous fundamental methods. For example, many approaches in frequent pattern mining have employed the data structures, pruning techniques, and pattern growth manners

of *Apriori* [2] and *FP-Growth* [12]. Such a tendency is also shown in the other pattern mining areas such as high utility pattern mining, representative pattern mining, and even in uncertain pattern mining that is the main focus on this paper. In other words, devising a well-designed algorithm has significant effects on developing advanced mining techniques and applications in wide areas. However, it is a difficult challenge to propose a novel efficient algorithm. In order that an algorithm is considered efficient, it has to guarantee faster runtime, smaller memory usage, and better scalability compared to state-of-the-art techniques, not to mention its accuracy. As more and more data have been accumulated in various areas according to the passage of time, algorithm performance has become more important. Therefore, if time to mine interesting patterns becomes too long, it can cause fatal problems such as failure of real-time data analysis and interactive responses to the mining requests of users. Memory problems such as memory overflow may be even worse than the runtime issues because they directly make algorithms fail to operate normally. In this regard, we need to consider designing a novel approach that is more efficient than previous state-of-the-art algorithms.

Motivated by the above issues, we propose a List-based UNcertain frequent pattern mining Algorithm (*LUNA*), which is an efficient and exact method for mining uncertain frequent patterns based on novel data structures and mining techniques that can guarantee the correctness of the mining results without any false positives. The main contributions of this paper are as follows:

- 1) Proposing a new paradigm for efficiently mining uncertain frequent patterns from uncertain databases.
- 2) Devising novel data structures based on list forms that can store uncertain data more efficiently compared to tree structures of previous approaches.
- 3) Proposing an algorithm that can extract exact results of uncertain pattern mining without any false positives using the suggested data structures.
- 4) Suggesting new pre-pruning techniques that can improve the mining performance of the proposed algorithm by preventing useless mining operations from being caused, and effective strategies that can speed up the mining operations without any additional memory consumption.

Extensive, comprehensive results of performance evaluation in this paper show that the proposed algorithm can mine exact uncertain frequent pattern mining results compared to previous state-of-the-art approaches and outperform them in terms of runtime, memory usage, and scalability.

The remainder of this paper is organized as follows. Section 2 introduces background knowledge for the proposed method, including various related works. Section 3 describes details of our approach, including the proposed novel data structure, mining techniques, and performance improving strategies. Section 4 shows

results of performance evaluation for proving that the proposed method outperforms previous state-of-the-art approaches. Section 5 discusses limitations and future work of the proposed method. Section 6 finally concludes this paper.

2. Related Work

In this section, we introduce the basic concept of frequent pattern mining and uncertain frequent pattern mining, and briefly describe the features of major related works. In addition, we explain the differences among the proposed algorithm and the relevant methods and describe the advantages and disadvantages of them.

2.1. Fundamental frequent pattern mining

Since the *Apriori* algorithm was proposed, a variety of frequent pattern mining approaches have been actively proposed [11, 12, 32]. The main purpose of fundamental frequent pattern mining is to find all of the possible meaningful patterns from a given database, where a threshold factor, called minimum support threshold, is employed to decide whether or not each of the found ones is interesting. In other words, a frequent pattern is defined as an itemset of which the support is higher than or equal to a given minimum support threshold. *FP-Growth* [12] is an algorithm devised to overcome the limitations of *Apriori*. Especially, a well-known tree structure for frequent pattern mining, called *FP-Tree*, and depth-first-search-based mining techniques were proposed in the literature, and they have been utilized as an important basic framework in various works [11, 14, 25, 40]. *FP-Growth* stores given transaction data into its own tree structure, *FP-Tree*, in a compressed form. In contrast to the case of the *Apriori* algorithm, the algorithm performs its mining operations within two database scans and extracts frequent pattern mining results without generating any candidate patterns.

The data structure and mining techniques of *FP-Growth* have given data mining researchers many inspirations and allowed them to develop a variety of variations [11, 32]. *FP** [11] is an algorithm that improves mining speed using an additional array structure, called *FP-array*. Although the algorithm spends larger memory compared to *FP-Growth* because it has to construct and maintain additional data structures, this method guarantees faster mining speed by reducing the number of conditional tree scans in the mining process. *LP-growth* [32] is a method for mining frequent patterns more efficiently through its own tree structure based on array components, called *LP-tree*, which is also one of the variations of *FP-Tree*. In addition, various tree-based pattern mining approaches have been proposed such as weighted pattern mining [14, 49, 50], utility pattern mining [35, 47, 51], representative pattern mining [45, 49], and stream pattern mining [46, 47].

However, since the aforementioned algorithms are approaches focusing on precise data, they have difficulty in dealing with uncertain data that cannot exactly be expressed as presence or absence according to characteristics

and environments of accumulated data. Meanwhile, the proposed algorithm is able to extract uncertain frequent patterns from such uncertain data in a more efficient manner.

2.2. Uncertain frequent pattern mining

Since the concept of uncertain pattern mining was proposed to discover interesting pattern information from uncertain databases, various relevant approaches have been devised [1, 20, 38, 40, 41]. In contrast to items in normal databases (or called transaction databases), items composing uncertain databases additionally have their own existential probability values. *U-Apriori* [7] was first proposed to mine uncertain frequent patterns. The algorithm follows the basic framework of the *Apriori* algorithm; hence, it is also affected by the fundamental performance limitations of *Apriori*. *MBP* [41] is an approximation method for uncertain frequent pattern mining based on statistical techniques. Although the algorithm is also an *Apriori*-based level-wise method, it can extract uncertain frequent patterns faster than *U-Apriori*. *IMBP* [38] was proposed to more improve the mining speed and memory efficiency of *MBP* at the cost of losing accuracy. However, the degree of accuracy losses in this method is significant in comparison to exact approaches. In particular, its accuracy is much lower on dense databases and accuracy results are not stable. These two algorithms still have drawbacks regarding excessive database scans and candidate creation. Furthermore, they cannot draw exact mining results from given uncertain databases since they are approximation approaches.

Mining techniques of *FP-Growth* and its data structure have given many inspirations to the uncertain pattern mining area as well as the traditional frequent pattern mining field. Unlike previous level-wise uncertain pattern mining methods, Tree-based depth-first search (DFS) approaches [19, 20, 22, 40] scan given uncertain databases only twice and construct tree structures for their mining processes to extract valid uncertain frequent patterns without any candidate generation. *UF-Growth* [20] is an initial tree-based algorithm that mines uncertain frequent patterns using its own tree structure, *UF-Tree*, a variation of *FP-Tree*. Unlike the case of *FP-Tree*, some information losses of existential probability values can be caused in the process of inserting uncertain data and merging nodes if transaction data in uncertain databases are inserted into the tree in the same manner as that of *FP-tree*. In *UF-Tree*, conditions for its node merging operations are set more strictly than those of *FP-Tree* in order to prevent such a side effect. That is, in *FP-Tree*, nodes with the same parent node and item name are allowed to merge with one another. Meanwhile, in *UF-Tree*, the targets of node merging operations become only nodes having the same existential probability as well as satisfying the above conditions. Based on these strict conditions, *UF-Growth* guarantees exact results of uncertain frequent pattern mining but suffers from performance degradation in terms of runtime and memory usage. Thereafter, another version of *UF-Growth* was

proposed to improve its mining performance [19]. However, its main strategy, which mitigates constraints of the node merging operations using a parameter, k -digit measure, can raise algorithm performance but cause pattern losses and false positives. That is, after k is set by a user, *UF-Growth* ignores more than k decimal places in existential probabilities of nodes, which makes more nodes merged with one another although their values are not actually the same. Therefore, it can cause pattern losses because of disregarded decimal point values. For this reason, although the algorithm does not generate any false negatives, it is still not an exact method since the k -digit measure may distort original existential probabilities as lower values, which can cause false positive problems (i.e., pattern losses) depending on the settings of k . As another uncertain approach, *UH-Mine* [1] mines uncertain frequent patterns using the basic framework of *H-Mine* [30], which is a fundamental frequent pattern mining algorithm. This non-tree-based method employs its own data structure, called *H-struct*. *UH-Mine* shows good results of mining performance on small-scale databases; meanwhile, its performance becomes worse on large-scale databases.

There are several literatures for mitigating that previous approximation methods mine not only valid pattern results but also enormous false positives. By using an overestimation method, called *Prefixed Item Cap (PIC)*, *PUF-growth* [18] constructs its own tree structure, named *PUF-Tree*, as compact as *FP-Tree*. Based on the constructed tree, the algorithm generates a smaller number of false positives compared to previous tree-based approximation methods. *TPC-growth* [17] is an advanced version of *PUF-growth*. It employs an upgraded overestimation method that can tighten upper bounds to expected supports more than *PUF-growth*. However, they still do not extract results of uncertain frequent patterns from given uncertain databases. On the other hand, the proposed algorithm always draws exact results considering uncertainty of the given data.

In order to overcome the aforementioned problems of initial tree-based uncertain pattern mining and obtain precise uncertain pattern mining results, the following approaches were proposed [22, 31]. *CUEP-Mine* [22] is a method for mining exact uncertain frequent patterns without employing recursive call-based pattern growth manners. Its own tree structure, called *CUEP-Tree*, includes an array list for each node in order to construct the tree as compact as *FP-Tree* without any loss of uncertainty information. The array list of each node stores existential probability values for all of the possible item combinations that can be generated from the node. Although such a method is effective on relatively high threshold settings, its performance degradation becomes significant at relatively low threshold settings because the algorithm has to store and manage the information of all the possible item combinations for each node. Moreover, the larger the given database is, the worse the mining performance of *CUEP-Mine* becomes. *AT-Mine* [40] is another tree-based efficient approach proposed to

overcome the fatal problems of *CUIP-Mine*. By applying the same level of constraints for node merging operations as those of *FP-Tree* into its tree structure, *AT-Tree*, the algorithm can express given uncertain data as a compact tree structure with the same number of nodes as that of *FP-Tree*. Furthermore, in order to prevent any loss of uncertainty information that can occur in the process of merging nodes, *AT-Mine* uses a global array structure, called *ProArr* (different from the arrays of *CUIP-Mine*), which stores transaction IDs (*TIDs*) and existential probability values of items within a given uncertain database. *AT-Mine* conducts *FP-Growth*-like mining operations from the constructed *AT-Tree* and *ProArr*. In this process, the algorithm divides nodes composing the tree into two types: normal node and tail node, and stores additional information into tail nodes in order to reference the uncertainty information of *ProArr* more efficiently. *AT-Mine* guarantees more efficient mining performance than that of *CUIP-Mine*, but it still has limitations in runtime and memory performance aspects. *U-WFI* [16] is a tree-based approach that applies weight factors into uncertain pattern mining. Through weight constraints, the algorithm can find more meaningful uncertain frequent patterns but have limitations in the aforementioned aspects. On the other hand, in this paper, we propose a new paradigm for uncertain frequent pattern mining, which can overcome the above problems and extract uncertain frequent patterns more efficiently. Table 1 shows the overall characteristics of various uncertain pattern mining algorithms.

Table 1. Characteristics of uncertain pattern mining approaches

Algorithm	Exact mining method	Tree-based method	Extraction of false positives	Note
<i>U-Apriori</i> [7]	Y	N	N	<i>Apriori</i> -based method
<i>MBP</i> [41]	N	N	Y	Use of statistical techniques
<i>IMBP</i> [38]	N	N	Y	Faster speed but lower accuracy than that of <i>MBP</i>
<i>UF-Growth</i> [19, 20]	Y/N	Y	Y/N	<i>FP-growth</i> -based method
<i>UH-Mine</i> [1]	Y	N	N	<i>H-mine</i> -based method
<i>PUF-Growth</i> [18]	N	Y	Y	Use of upper bounds to expected supports
<i>TPC-Growth</i> [17]	N	Y	Y	Use of more tightened upper bounds
<i>CUIP-Mine</i> [22]	Y	Y	N	Non-growth method
<i>AT-Mine</i> [40]	Y	Y	N	<i>FP-growth</i> -based method
<i>U-WFI</i> [16]	Y	Y	N	Weight-based approach
<i>LUNA</i>	Y	N	N	The proposed method

Note that comparable algorithms among the above ones are *CUIP-Mine* and *AT-Mine* because they are state-of-the-art methods that can mine exact results of uncertain frequent pattern mining. The others are excluded in our performance evaluation tests since they are too old (*Apriori*-based approaches), cannot extract exact mining results (approximation methods), or cannot conduct reasonable performance comparisons (a weight-based approach).

3. Mining Uncertain Frequent Patterns Using Minimum Data Structure without False Positives

In this section, we propose a new paradigm for uncertain frequent pattern mining and describe details of the proposed method, *LUNA*. We first introduce a problem definition of uncertain frequent pattern mining and relevant mathematical preliminaries. After that, we explain novel data structures for conducting uncertain frequent pattern mining operations efficiently and how to construct the proposed data structures. Thereafter, we describe an algorithm that performs mining operations employing the devised data structures without any false positive. We also provide rich empirical examples to help understand the contents of the proposed method.

3.1. Problem definition and mathematical preliminaries

Unlike normal databases used in fundamental frequent pattern mining, an uncertain database for uncertain frequent pattern mining can be denoted as follows. Let $D = \{T_1, T_2, \dots, T_n\}$ be a given uncertain database with multiple transactions, T_s , $T_x = \{i_1, i_2, \dots, i_l\}$ ($1 \leq x \leq n$) be a transaction belonging to D , and $I = \{i_1, i_2, \dots, i_m\}$ be a set of distinct items in D , where each T is a sub set of I . Then, each item i_j in $T_x = \{i_1, i_2, \dots, i_l\}$ ($1 \leq j \leq l$) can belong to two possible worlds, *existence* (denoted as W_1 and $i_j \in T_x$ in W_1) and *non-existence* (denoted as W_2 and $i_j \in T_x$ in W_2), respectively. Let $p(i_j, T_x)$ be an existential probability of i_j in T_x ($0 \leq p(i_j, T_x) \leq 1$). Then, the value that i_j is included in either W_1 or W_2 can be denoted as $p(i_j, T_x)$ and $1 - p(i_j, T_x)$, respectively. In a similar way, we can express an existential probability of a pattern composed of two or more items. Given two items, i_j and i_k in T_x ($1 \leq j < k \leq x$), the probability value that i_j and i_k belong to W_1 or W_2 can be expressed as $p(i_j, T_x) \times p(i_k, T_x)$ and $(1 - p(i_j, T_x)) \times (1 - p(i_k, T_x))$, respectively. From the characteristics, we can denote the relationship between W_1 and W_2 for T_x as $W_1 \cup W_2 = T_x$. Similarly, the relationship between W_1 and W_2 for D is also denoted as $W_1 \cup W_2 = D$.

Table 2 shows an example of an uncertain database. Using the database types, uncertain frequent pattern mining algorithms extract uncertain frequent patterns. As shown in the table, each item has a unique existential probability value between 0 (0%) and 1 (100%). Note that settings of existential probability values for the same item can be different for each transaction as shown in the table.

Table 2. Example of an uncertain database

TID	Items							
	A	B	C	D	E	F	G	H
100	1.0	-	0.9	0.6	-	-	-	-
200	0.9	0.9	0.7	0.6	0.4	-	-	-
300	-	0.5	0.8	0.9	-	0.2	0.4	-
400	-	-	0.9	-	0.1	0.5	-	0.8
500	0.4	0.5	0.9	0.3	-	-	0.3	0.3
600	-	-	-	0.9	0.1	0.6	-	0.3
700	0.9	0.7	0.4	0.6	-	0.9	-	-

Definition 1. (Expected support of a pattern (expSup)) Let $X = \{i_1, i_2, \dots, i_s\}$ be a pattern composed of items belonging to I . Then, the existential probability of X for each transaction T_x , $p(X, T_x)$, is computed as follows:

$$p(X, T_x) = \prod_{i_k \in T_x} p(i_k, T_x). \quad (1)$$

In Equation (1), T_x is limited as the ones having all of the items in X among all the transactions in D . The expected support of X , $expSup(X)$, is then calculated as follows:

$$expSup(X) = \sum_{T_x \in D} p(X, T_x) = \sum_{T_x \in D} \left\{ \prod_{i_k \in T_x} p(i_k, T_x) \right\}. \quad (2)$$

In order to calculate an expected support, the factor for judging validity of each mined pattern in uncertain frequent pattern mining, we need additional considerations as shown in Equation (2), which is different from the support calculation manner in traditional frequent pattern mining. In other words, the result of Equation (2) is to add all of the existential probabilities of pattern X from every transaction with the pattern.

Definition 2. (Uncertain frequent pattern (UFP)) Let δ be a threshold percent value given by a user and n be the total number of transactions in a given uncertain database, D . Then, a user-specified minimum support threshold, $minSup$, is computed by multiplying δ by n . If the expected support of pattern X , $expSup(X)$, is higher than or equal to $minSup$, X becomes an uncertain frequent pattern, UFP .

As in the case of the support factor in fundamental frequent pattern mining, the expected support factor also satisfies the anti-monotone property that reduces the search space for mining operations without any pattern loss.

Property 1. Let $X = \{i_1, i_2, \dots, i_j\}$ be an uncertain pattern and $Y = \{i_1, i_2, \dots, i_k\}$ be a super pattern of X such that $j < k$ and $X \subset Y$. Then, if $expSup(X) < minSup$, it is also true that $expSup(Y) < minSup$.

Proof. As shown in Equation (2), $expSup$ of a pattern is to 1) calculate the corresponding existential probability of the pattern for each transaction including all of the items within the pattern and 2) added all of the calculated results. Therefore, this equation can be considered as two aspects: 1) calculating an existential probability value of a specific pattern from a transaction and 2) adding all of the computed values. In the first aspect, an existential probability value of a specific pattern is to multiply the existential probability of each item within the pattern in sequence. Since any probability value cannot exceed 1, it is obvious that the existential probability of a pattern becomes lower or maintains the same level as its length becomes longer. That is, the existential probability of X for each transaction is always higher than or equal to that of Y because $j < k$ and $X \subset Y$. In the second aspect, we already know through the characteristics of the support factor [2, 12] that the number of transactions including a specific pattern becomes smaller or maintains the same level as the pattern length becomes longer. That is, the number of existential probability values added to compute $expSup(X)$ is always larger than or equal to that of $expSup(Y)$. We can determine from these characteristics that, if $expSup(X) < minSup$

$minSup, expSup(Y) < minSup$ because $expSup(X) < expSup(Y)$. ■

Example 1. For the items of the example database in Table 1, A, B, C, D, E, F, G , and H , their expected support values are calculated as follows: $expSup(A) = 1.0 + 0.9 + 0.4 + 0.9 = 3.2$, $expSup(B) = 0.9 + 0.5 + 0.5 + 0.7 = 2.6$, $expSup(C) = 0.9 + 0.7 + 0.8 + 0.9 + 0.9 + 0.4 = 4.6$, $expSup(D) = 0.6 + 0.6 + 0.9 + 0.3 + 0.9 + 0.6 = 3.9$, $expSup(E) = 0.4 + 0.1 + 0.1 = 0.6$, $expSup(F) = 0.2 + 0.5 + 0.6 + 0.9 = 2.2$, $expSup(G) = 0.4 + 0.3 = 0.7$, and $expSup(H) = 0.8 + 0.3 + 0.3 = 1.4$. If the user-given minimum support threshold is 0.3 (30%), $minSup$ becomes 2.1. Then, items E, G , and H become invalid ones because their $expSup$ values are smaller than $minSup$. Let $X = \{A, B\}$ be a pattern that can be obtained from the example database. Then, $expSup(X) = \{(0.9 * 0.9) + (0.4 * 0.5) + (0.9 * 0.7)\} = 1.64 < minSup$. Hence, X becomes an uncertain infrequent pattern. Meanwhile, for another pattern $Y = \{C, D\}$, $expSup(Y) = \{(0.9 * 0.6) + (0.7 * 0.6) + (0.8 * 0.9) + (0.9 * 0.3) + (0.4 * 0.6)\} = 2.19 > minSup$. Therefore, Y becomes an *UFP*.

Problem statement. Given an uncertain database, D , and a user-specified minimum support, $minSup$, mining a complete set of *UFPs* is equivalent to extract from D all of the possible patterns of which the expected supports are not smaller than $minSup$.

The main focus of the proposed algorithm is to solve the aforementioned problem more efficiently though the suggested novel data structures and mining techniques. In the subsequent sections, we describe their details including various empirical examples.

3.2. Overall architecture of the proposed method

Fig. 1 shows the overall architecture of the proposed algorithm. Given an uncertain database, the proposed algorithm, *LUNA*, scans the data twice in order to construct the proposed data structure, *UP-List*. In the first database scan, the algorithm calculates $expSup$ for each item belonging to the given database. After discarding invalid items of which the $expSup$ values are lower than $minSup$ given by a user, the algorithm computes a support ascending order for the remaining items. Thereafter, the algorithm scans the database again to construct and update *UP-Lists* based on the result of the first database scan, where the items corresponding to the *UP-Lists* become 1-length *UFPs*. After that, using the generated *UP-Lists*, our method recursively constructs conditional *UP-Lists*, called *CUP-Lists*, in order to extract *UFPs* with longer lengths. In this pattern growth process, various pruning techniques and speed-up techniques newly proposed in this paper are employed to improve the mining performance more effectively. After all of the mining processes of *LUNA* are finished, we can obtain a complete set of *UFPs* without any pattern loss and false positive.

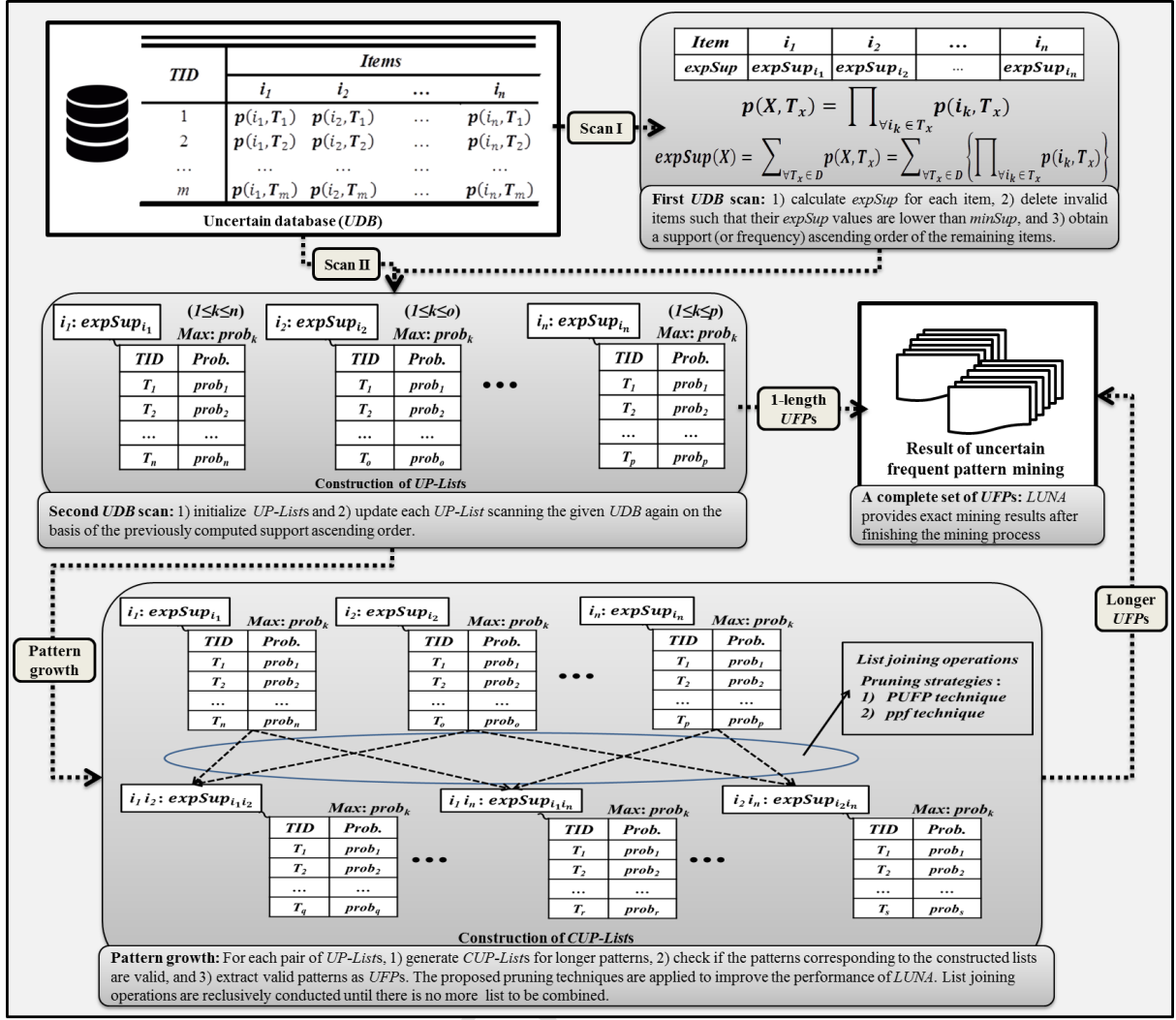
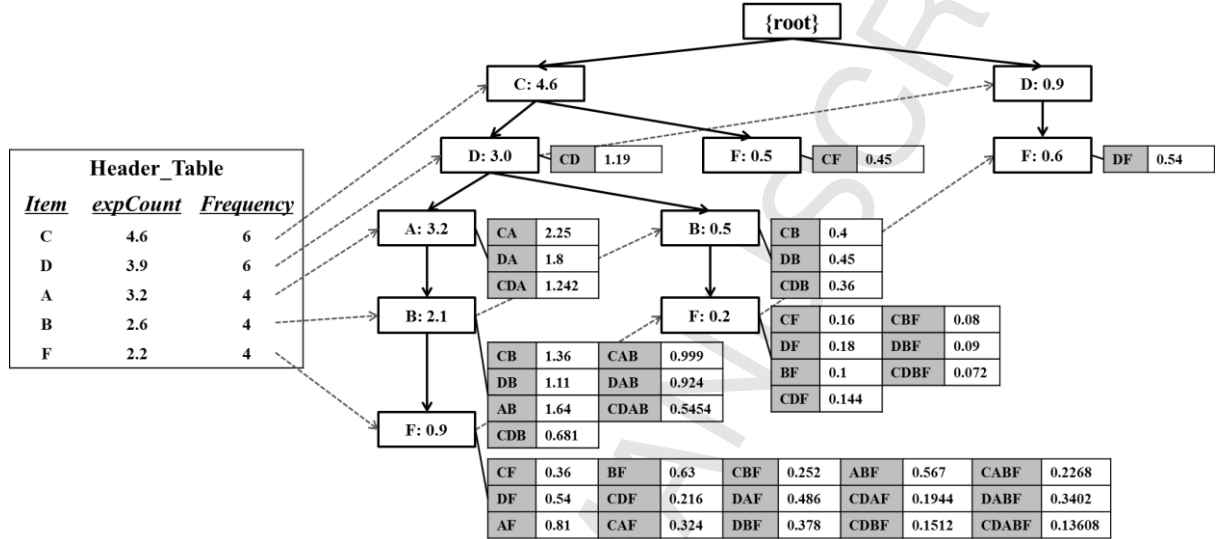


Fig. 1. Overall architecture of LUNA

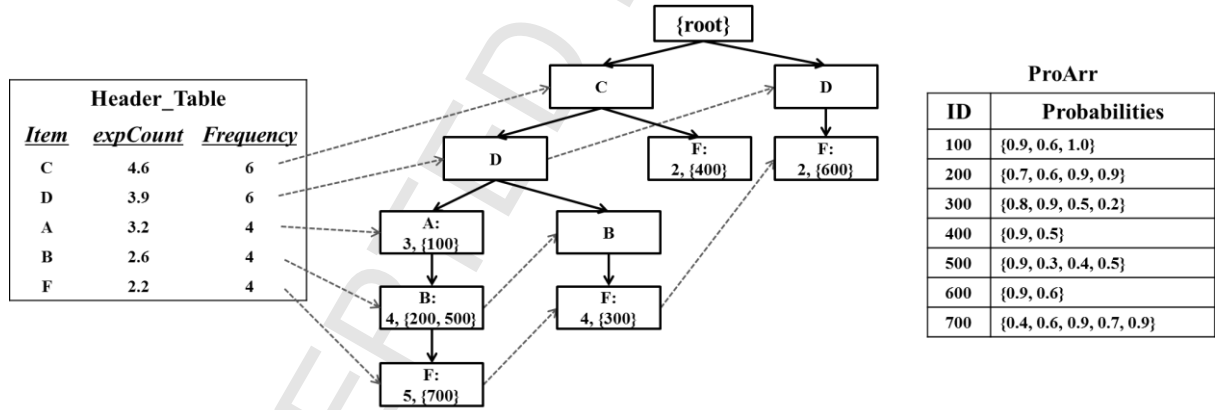
3.3. Minimum data structure for uncertain frequent pattern mining

In this section, we propose new data structures for conducting uncertain frequent pattern mining operations more efficiently and describe how to store uncertain data into the proposed data structures. Fig. 2 shows *CUFPTree* and *AT-Tree* used in the *CUFPT-Mine* and *AT-Mine* algorithms, respectively, when $minSup$ is 2.1 ($= 7 * 0.3$ (30%)) for the example database in Table 1. Although *CUFPTree* can express the given uncertain data within the same number of nodes as that of *FP-Tree*, It has to use an additional array structure for each node except for the child nodes of the root in order to maintain and manage the uncertainty information as shown in Fig. 2(a). Moreover, we can see that necessary array size becomes exponentially larger as the length of the corresponding path becomes longer. Such a feature causes critical degradation of mining performance. On the other hand, *AT-Tree* in Fig. 2(b) expresses the same data more simply than *CUFPTree*. It includes a global array list, *ProArr*, which is constructed to maintain and manage the given uncertainty information. In addition, the *AT-Mine* algorithm divides nodes of its tree into two node types: normal node and tail node, and stores additional

information into tail nodes in order to prevent any uncertainty information from being lost and utilize the information of *ProArr* effectively. Although *AT-Tree* can store uncertain data more efficiently than *CUFP-Tree* as shown in the figures, it still has limitations that should store and manage additional information such as *ProArr* and tail nodes and has to maintain many factors for constructing *AT-Tree* such as parent and child pointers, node-links, and a header table. Moreover, performance degradation caused by these problems can become more significant as a given uncertain database becomes sparser and its size becomes larger.



(a) *CUFP-Tree* constructed from Table 2 ($minSup = 2.1$)



(b) *AT-Tree* constructed from Table 2 ($minSup = 2.1$)

Fig. 2. Comparison of *CUFP-Tree* and *AT-Tree* on the same uncertain database and threshold setting

In order to overcome such problems and mine *UFPs* using minimum data structures and mining operations, we propose list-based novel data structures for uncertain frequent pattern mining.

Definition 3. (Uncertain probability-list (UP-List)) Given an uncertain database, D , the proposed *UP-List* is constructed through two scans of D . After finding uncertain frequent items (also called 1-length *UFPs*) through the first database scan, the proposed algorithm generates an *UP-List* for each found item. The list stores a pair of

the name and $expSup$ for the corresponding item and a set of tuples including TID information of the transactions with the current item and the corresponding existential probabilities. In the process of the second database scan, the tuple set and the maximum value of the probabilities are continually updated. The constructed UP -Lists are sorted in a support ascending order (the reason is explained in the subsequent section).

Fig. 3 shows the UP -Lists constructed from the example database in Table 1 when $minSup$ is 2.1. Once the UP -List construction processes are finished, the proposed algorithm extracts a complete set of UFP s through these minimum data structures by combining the lists in a recursive divide-and-conquer manner.

F: 2.2

Max: 0.9

TID	Prob.
300	0.2
400	0.5
600	0.6
700	0.9

B: 2.6

Max: 0.9

TID	Prob.
200	0.9
300	0.5
500	0.5
700	0.7

A: 3.2

Max: 1.0

TID	Prob.
100	1.0
200	0.9
500	0.4
700	0.9

D: 3.9

Max: 0.9

TID	Prob.
100	0.6
200	0.6
300	0.9
500	0.3
600	0.9
700	0.6

C: 4.6

Max: 0.9

TID	Prob.
100	0.9
200	0.7
300	0.8
400	0.9
500	0.9
700	0.4

Fig. 3. UP -Lists constructed from Table 2 ($minSup = 2.1$)

Example 2. Fig. 4 shows how to construct UP -Lists from the example uncertain database in Table 1 when $minSup = 2.1$. First, $LUNA$ scans the given uncertain database and selects valid items of which the $expSup$ values are higher than or equal to the given $minSup$, 2.1. In this process, meaningless items E , G , and H are excluded because their $expSup$ values are all smaller than $minSup$ ($expSup(E) = 0.6$, $expSup(G) = 0.7$, and $expSup(H) = 1.4$). Fig. 4(a) shows the states of the UP -Lists after the first transaction is processed. Initial UP -Lists have their own item name and $expSup$ values (they are obtained through the first database scan). The first transaction after deleting invalid items and sorting the remaining ones in a support ascending order is $\{A: 1.0, D: 0.6, C: 0.9\}$. Then, the tuple information of the corresponding UP -Lists is updated and their own Max values are changed as shown in the figure. After that, the same operations are performed for the second transaction as shown in Fig. 4(b). Fig. 4(c) shows the states of the UP -Lists when the algorithm processes the transactions up to $TID: 300$. As shown in the figure, the states of the lists are continually updated whenever each of new transactions is processed. Fig. 4(d) is the result of the UP -Lists after the transactions of the example database are inserted up to $TID: 500$. If all the transactions in Table 1 are inputted into the lists after refining each of them, we can obtain the final result of UP -Lists shown in Fig. 3.

F: 2.2	Max: 0	B: 2.6	Max: 0	A: 3.2	Max: 1.0	D: 3.9	Max: 0.6	C: 4.6	Max: 0.9
TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.
				100	1.0	100	0.6	100	0.9

(a) *UP-Lists* after inserting the transaction of *TID* 100

F: 2.2	Max: 0	B: 2.6	Max: 0.9	A: 3.2	Max: 1.0	D: 3.9	Max: 0.6	C: 4.6	Max: 0.9
TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.
		200	0.9	100	1.0	100	0.6	100	0.9
				200	0.9	200	0.6	200	0.7

(b) *UP-Lists* after inserting the transactions up to *TID* 200

F: 2.2	Max: 0.2	B: 2.6	Max: 0.9	A: 3.2	Max: 1.0	D: 3.9	Max: 0.9	C: 4.6	Max: 0.9
TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.
300	0.2	200	0.9	100	1.0	100	0.6	100	0.9
		300	0.5	200	0.9	200	0.6	200	0.7
						300	0.9	300	0.8

(c) *UP-Lists* after inserting the transactions up to *TID* 300

F: 2.2	Max: 0.5	B: 2.6	Max: 0.9	A: 3.2	Max: 1.0	D: 3.9	Max: 0.9	C: 4.6	Max: 0.9
TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.	TID	Prob.
300	0.2	200	0.9	100	1.0	100	0.6	100	0.9
400	0.5	300	0.5	200	0.9	200	0.6	200	0.7
		500	0.5	500	0.4	300	0.9	300	0.8
						500	0.3	400	0.9
								500	0.9

(d) *UP-Lists* after inserting the transactions up to *TID* 500

Fig. 4. Construction process of *UP-Lists*

Through the proposed novel minimum data structure, we can effectively express given uncertain data. Once the *UP-List* construction processes are finished, the proposed algorithm mines a complete set of *UFPs* using the constructed lists in the following recursive divide-and-conquer manner.

3.4. Mining *UFPs* from the proposed *UP-Lists*

In the previous section, we introduced details of the proposed *UP-List*, the minimum data structure for mining *UFPs* more efficiently, and explained how to construct *UP-Lists* from a given uncertain database. In this section, we describe how the proposed algorithm can effectively mine a complete set of exact *UFPs* without any false positives from the constructed *UP-Lists*.

3.4.1. Recursive construction of conditional *UP-Lists* for mining *UFPs*

Once *UP-Lists* are constructed from a given uncertain database, the proposed algorithm extracts *UFPs* with longer lengths for each *UP-List* in a recursive divide-and-conquer manner. In other words, we first select an *UP-List* among the constructed ones and then set the corresponding item of the list as a prefix. Thereafter, we choose another *UP-List* to set a pair of *UP-Lists* and start to construct conditional *UP-Lists* from the *UP-List* pair. If the

$expSup$ value of the pattern corresponding to a constructed conditional *UP-List* is smaller than $minSup$, all of the tasks related to the conditional list are discarded by the anti-monotone property in Property 1. Otherwise, the pattern is extracted as a valid *UFP*. If all of the possible conditional *UP-Lists* are constructed from the current prefix, the proposed algorithm expands the prefix by adding an item and recursively generates conditional lists from the expanded prefix in a depth-first search manner in order to mine *UFPs*.

Definition 4. (Conditional uncertain probability-list (CUP-List)) Unlike the case of the *UP-List* with an item name and its $expSup$ value, A *CUP-List* includes a pattern name composed of two or more items, its $expSup$ value, a set of tuples with TID information of the transactions containing the pattern and the corresponding existential probability information. The *CUP-List* construction method is divided into the following two cases depending on the current state of the mining process.

Case 1. Given two *UP-Lists*, U_1 and U_2 , a *CUP-List* for them is constructed as follows. Let i_1 and i_2 be items of U_1 and U_2 , respectively. Then, the pattern name of the *CUP-List* becomes $\{i_1, i_2\}$ and its $expSup$ value is computed by Equation (2), where the tuple sets stored in the *UP-Lists* are used to calculate $expSup$ effectively without additional database scans. In the process of computing $expSup$, tuple information of the current *CUP-List* is also continually updated; at the same time, its Max value is also updated.

Case 2. Let C_1 and C_2 be two *CUP-Lists* where their own patterns have the same length, and $X = \{i_1, i_2, \dots, i_{k-1}, x\}$ and $Y = \{i_1, i_2, \dots, i_{k-1}, y\}$ be the patterns of C_1 and C_2 , respectively ($k > 1$). Then, prefix becomes the common part between X and Y : $\{i_1, i_2, \dots, i_{k-1}\}$, and the pattern of the constructed *CUP-List* becomes $XY = \{i_1, i_2, \dots, i_{k-1}, x, y\}$. After that, $expSup$ of XY is computed and the tuple information and the max value for the *CUP-List* are updated as in Case 1.

Recall that the proposed algorithm generates *CUP-Lists* for patterns with longer lengths through combinations of *UP-Lists* or *CUP-Lists* with the same pattern length in order to mine valid *UFPs* efficiently. Therefore, the following characteristics are always satisfied when two lists are combined in the mining process.

Lemma 1. Let C_1 and C_2 be given two *CUP-Lists* for patterns with the same length and $C_1.pattern = \{x_1, x_2, \dots, x_k\}$ and $C_2.pattern = \{y_1, y_2, \dots, y_k\}$ be the patterns of C_1 and C_2 , respectively. Then, it is always true that $x_1 = y_1, x_2 = y_2, \dots, x_{k-1} = y_{k-1}$. Hence, for a *CUP-List* for a longer pattern generated from them, C' , the pattern of C' , $C'.pattern$ becomes $\{x_1, x_2, \dots, x_k, y_k\}$.

Proof. Let U_j ($1 \leq j \leq m$; m : the number of uncertain frequent items) be an *UP-List* with its own item, i_j (denoted as $U_j.item$). Then, each *CUP-List* constructed from $L_j, C_{j,k}$ ($j < k; 1 \leq k \leq m - j$), has pattern $\{i_j, i_k\}$ (denoted as

$C_{j,k}.pattern$) because the current prefix item is i_j and i_k is a changeable item in this phase. After that, each *CUP-List* generated from $C_{j,k}$, $C_{j,k,l}$ ($k < l$; $1 \leq l \leq m - j - k$), has pattern $\{i_j, i_k, i_l\}$, where i_j and i_k are always the items of the prefix while i_l is a changeable one because every $C_{j,k,l}$ must be constructed on the basis of $C_{j,k}$. In a similar way, constructions of subsequent *CUP-Lists* from $C_{j,k,l}$ follow the above characteristics. From the features of recursive creations for *CUP-Lists*, we can effectively obtain pattern names of *CUP-Lists* without any complicated considerations and calculations. ■

When generating *CUP-Lists* for longer patterns from the previously constructed *UP-Lists* or *CUP-Lists*, we can easily know their pattern names by the above lemma. After that, calculations for obtaining their *expSup* values are performed. Recall that the proposed data structures, *UP-Lists* and *CUP-Lists*, have their own tuple sets that include *TID* information of transactions with the corresponding pattern and existential probability information for the pattern. From this useful information, we can easily calculate *expSup* values of patterns. Let us consider Case 1 of Definition 4 first. Given two *UP-Lists*, U_1 and U_2 , a *CUP-List* that can be generated from them, C , is constructed as follows. $C.pattern = U_1.item \cup U_2.item = \{i_1, i_2\}$ depending on Definition 4. $C.expSup$ is computed as follows. For the tuple sets in U_1 and U_2 , we find *TIDs* belonging to both the tuple sets, where the corresponding existential probability values are multiplied whenever each *TID* satisfying the condition is found. The computed results are added to the *expSup* value of the current pattern. In the process of adding the *expSup* value, the tuple information of C is also updated. After iterating the above works with respect to all of the *TIDs*, we can obtain the exact result of *expSup*. Next, let us consider Case 2 of Definition 4. Given two *CUP-Lists*, C_1 and C_2 , a *CUP-List* for a longer pattern that can be generated from them, C' , is constructed as follows. The pattern name of C' , $C'.pattern$, can easily be obtained from the pattern names of C_1 and C_2 as shown in Lemma 1. However, in order to calculate *expSup* of C' exactly, additional considerations are needed unlike Case 1 of Definition 4. In Case 1, existential probabilities within the tuple sets of the *UP-Lists* are the values for the corresponding items. Therefore, we can simply calculate intended *expSup* values by multiplying and adding these probability values. Meanwhile, in Case 2 that generates a *CUP-List* for a longer pattern from two existing *CUP-Lists*, existential probabilities stored in the tuple sets of C_1 and C_2 are probability values that consider all the items composing the patterns of C_1 and C_2 including the prefix part. Therefore, if we follow the calculation manner of Case 1 when processing the existential probabilities in C_1 and C_2 , existential probability values corresponding to the items of the current prefix are considered twice. Accordingly, we cannot draw exact results in this manner. Hence, in order to calculate *expSup* of C' exactly without any duplication, we multiply each of the existential probabilities in the tuple set of C_1 by each of the existential probabilities corresponding to the last

item of the C_2 's pattern and then accumulate the calculated results. In the process of updating $expSup$, tuple set information of C' is also updated as in Case 1. Through the above method, we can construct correct CUP -Lists in Case 2.

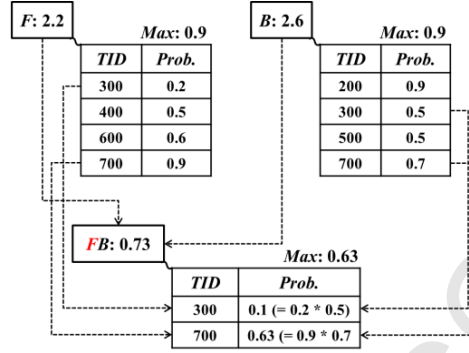
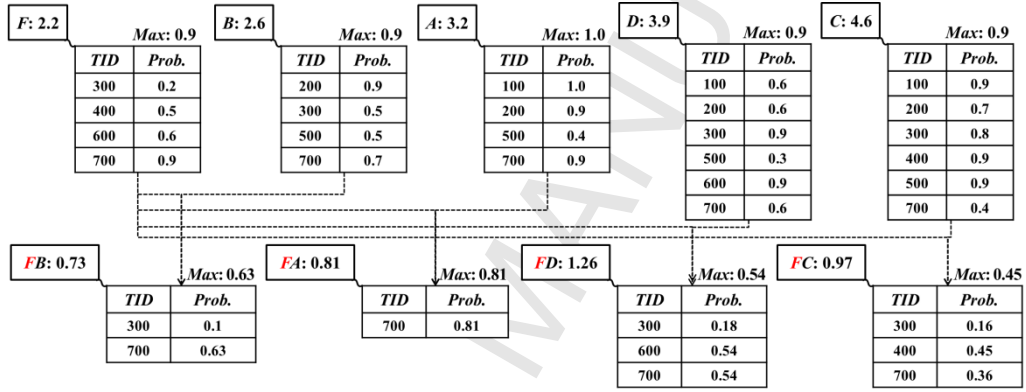
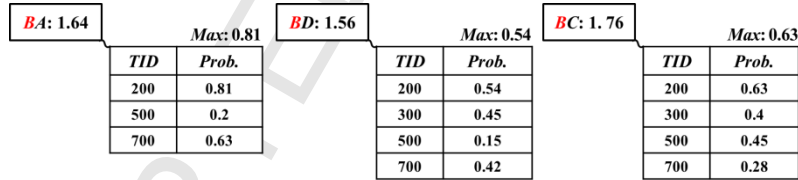


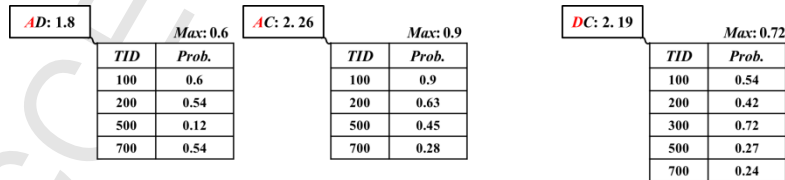
Fig. 5. Construction process of the CUP -List for FB using the UP -Lists for F and B in Fig. 3



(a) CUP -Lists constructed on the basis of the UP -List for F



(b) CUP -Lists constructed on the basis of the UP -List for B



(c) CUP -Lists constructed on the basis of the UP -Lists for A and D

Fig. 6. CUP -Lists generated from the UP -Lists in Fig. 3

Example 3. Consider how to construct CUP -Lists from the UP -Lists shown in Fig. 3 when $minSup$ is 2.1. Fig. 5 shows how to construct a CUP -List for pattern FB that can be generated from the first two UP -Lists for item F

and B in Fig. 3. First, we store the combination of the UP -Lists' item names into a new CUP -List. After that, we compare the tuple sets of the UP -Lists for F and B to find where they have common parts with the same TID information. As a result, TID : 300 and 700 are considered to compute $expSup$ of FB . As shown in the figure, we can determine that the result of FB 's $expSup$ is 0.73 by adding the product of the existential probabilities corresponding to TID : 300 and that of TID : 700. However, since the value is lower than the given $minSup$, FB cannot become an UFP . Fig. 6(a) shows the results of UP -List combinations based on F 's UP -List. However, all of them cannot become UFP s because their $expSup$ values are smaller than $minSup$. Other CUP -Lists are also constructed in the same manner as shown in Figs. 6(b) and (c). From the results, we can determine that valid UFP s obtained from the given example uncertain database and the $minSup$ setting are F , B , A , D , C , AC , and DC . Recall that the proposed algorithm recursively constructs CUP -Lists from the UP -Lists made from a given uncertain database in a depth-first-search manner. However, in the above example, the algorithm does not recursively generate CUP -Lists because there is no case that creates two or more CUP -Lists in the process of combining the UP -Lists.

Example 4. Consider mining UFP s from the example uncertain database in Table 1 when $minSup = 1.4$ ($= 7 * 0.2$ (20%)) in order to show recursive processes of the CUP -List construction. Then, in addition to the valid items shown in Fig. 3, item H also becomes an uncertain frequent item. Fig. 7 shows how to recursively construct CUP -Lists from the example database. Note that, since the CUP -Lists generated from the UP -Lists for items H , F , D , and C cannot satisfy the conditions for constructing CUP -Lists for longer patterns, their CUP -List results are excluded in the figure for a simple and clear expression. In other words, any CUP -List that can be constructed from the UP -Lists for H and F is invalid, and only one valid CUP -List is created from the UP -List for D . In the case of C , there is no CUP -List generated from the C 's UP -List. Once the three CUP -Lists for BA , BD , and BC are constructed on the basis of the UP -List for B , one of them is selected again and CUP -Lists for longer patterns are created on the basis of the newly selected one as shown in the figure. In other words, if the CUP -List for BA is selected, CUP -Lists for BAD and BAC are recursively made. Recall that there are additional considerations in order to obtain exact $expSup$ and tuple set information of CUP -Lists recursively constructed from other CUP -Lists as mentioned above. For this reason, the $expSup$ and existential probability values of the CUP -Lists for BAD and BAC are calculated as shown in the figure. However, their $expSup$ values are smaller than the given $minSup$; they cannot become UFP s. Moreover, since the number of valid CUP -Lists in the current state is less than 2, we do not need to conduct subsequent recursive operations. Thereafter, mining operations based on the CUP -List for BD are performed. As a result, the CUP -List for BDC is generated, but its

$expSup$ is lower than $minSup$. Accordingly, BDC does not become a valid pattern. The CUP -List for ADC can be generated from the CUP -List for AD by combining it with the CUP -List for AC . However, it is not an UFP because of its $expSup$ lower than $minSup$.

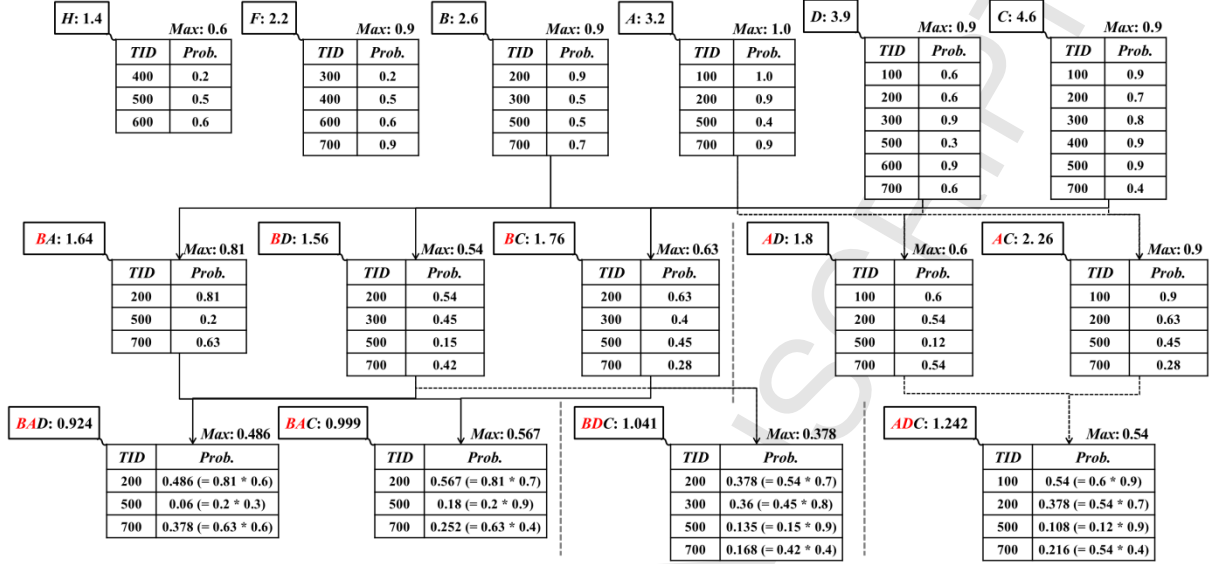


Fig. 7. Recursive processes of CUP -List constructions from Table 2 ($minSup = 1.4$)

By repeating the aforementioned works until there is no further UFP s to be extracted, we can obtain a complete set of UFP s considering exact existential probabilities and $expSup$ values of patterns.

In the subsequent sections, we describe various techniques that can dramatically improve the mining performance of the proposed algorithm such as pre-pruning techniques for reducing the search space and redundant mining operations and speedup techniques of the proposed method without additional use of memory. Meanwhile, recall that the proposed data structures are sorted in a support ascending order of items as shown in the above relevant figures. The reason why we employ the support ascending order, not the other orders such as a lexicographic order and a support descending order, is that sorting our data structures in this support ascending order allows the proposed algorithm to mine UFP s more efficiently in comparison to the others.

Lemma 2. Sorting UP -Lists in a support ascending order of uncertain frequent items allows the proposed algorithm to construct a smaller number of CUP -Lists in the mining process.

Proof. The proposed algorithm, *LUNA*, constructs UP -Lists and then recursively generates CUP -Lists from them to extract a complete set of UFP s from a given uncertain database. Let $U = \{U_1, U_2, \dots, U_n\}$ be a set of UP -Lists that can be generated from a given uncertain database and a threshold setting. Then, the maximum number of CUP -Lists that can be generated from U_1 becomes $n - 1$, that of U_2 becomes $n - 2$, ..., that of U_{n-1} becomes 1. Let $C_{U_1} = \{C_1, C_2, \dots, C_{n-1}\}$ be a maximum set of CUP -Lists that can be generated from U_1 . Then, $n - 2$ CUP -Lists can maximally be generated from C_{U_1} . From the above relations, we can determine that a larger

number of *CUP-Lists* can be constructed from *UP-Lists* located closer to the front of the *UP-List* sorting order (the sequence of *CUP-Lists* is decided by that of *UP-Lists*). We already know from the anti-monotone property that all the super patterns of any item or pattern always have *expSup* values smaller than or equal to that of the item or pattern. Hence, it is obvious that patterns generated from ones with smaller *expSup* values have much smaller *expSup* values. Then, they are more likely not to satisfy the given *minSup* constraint compared to patterns generated from items or patterns with relatively high *expSup* values. That is, by sorting *UP-Lists* in a support ascending order and finding invalid combinations in advance, we can minimize the number of *CUP-Lists* generated in the mining process and exclude meaningless operations in advance. ■

3.4.2. Pre-pruning techniques for reducing the search space and redundant mining operations

In this section, we propose pre-pruning techniques that can minimize the search space and redundant mining operations by preventing *CUP-Lists* causing meaningless pattern generation from being constructed.

A naïve manner in the proposed method is to 1) construct a complete *CUP-List* from given *UP-Lists* or *CUP-Lists* first, as shown in Figs. 6 and 7, and 2) check whether or not the corresponding pattern is a *UFP* by comparing its *expSup* value with *minSup*. However, such a manner has a drawback that has to construct all of the possible *CUP-Lists* for exact *UFP* mining results even though parts of the lists are actually useless. In addition, it may be unrealistic if a given uncertain database is very large and *minSup* is very low. To overcome this problem and improve the mining performance, we first propose a simple but strong pruning technique.

Definition 5. (Potential uncertain frequent pattern (PUFP)) Let $X = \{i_1, i_2, \dots, i_k\}$ be an *UFP* and i' be an item to be inserted into X . Then, a supper pattern of X , X' , can be denoted as $X' = \{i_1, i_2, \dots, i_k, i'\}$, and its *expSup* is computed as shown in Equation (2). Meanwhile, the overestimated *expSup* value of X' can be considered as follows. Let $Max(i')$ be the maximum value among the existential probabilities that i' can have in the given uncertain database. Then, the overestimated *expSup* of X' is calculated as $expSup(X) * Max(i')$. That is, this value is the maximum *expSup* value that X' can have. Hence, if the value is not smaller than *minSup*, X' becomes a potential uncertain frequent pattern (*PUFP*); otherwise, it becomes a permanently useless one.

The characteristics of Definition 5 are effectively utilized to check whether or not each of *CUP-Lists* is worth constructing completely when they are recursively created in our mining process. In other words, when there are two given *UP-Lists*, we can easily calculate the overestimated *expSup* value of the pattern that can be generated from the given lists through the *Max* information stored in each list (Recall that, when *UP-Lists* or *CUP-Lists* are constructed, the corresponding *Max* values are stored together). While *expSup* of a pattern should be computed though complicated calculation processes as shown in Equation (2), its overestimated *expSup* can

easily be obtained as shown in Definition 5. If a pattern obtained from given two *UP-Lists* is not a *PUFP*, we can omit all of the works related to constructing a *CUP-List* for the pattern.

Example 5. Consider applying the aforementioned overestimation method in the process of the *CUP-List* construction in Fig. 6(a). In order to construct a *CUP-List* for pattern *FB* from the *UP-Lists* for items *F* and *B*, we have to store the *CUP-List*'s pattern name, *expSup* value, *Max* value, and a set of tuples for recording *TID* and existential probability information related to the pattern. In particular, a series of processes for obtaining an *expSup* value including updates of the corresponding tuple information can cause more computational overheads compared to the other tasks. Furthermore, since the *expSup* value of the *CUP-List* for *FB* obtained through the above processes is 0.73 ($< minSup$), we can know that the pattern eventually becomes a useless one. On the other hand, since the overestimated *expSup* value is 1.98 ($= expSup(F) * Max(B) = 2.2 * 0.9$), we can directly determine that the pattern, *FB*, is not a *PUFP*; therefore it cannot also become an *UFP*. Accordingly, we can omit all of the operations for constructing the *CUP-List* for *FB*. Similarly, operations for generating *CUP-Lists* for *FD* and *FC* can be omitted. As a result, we can reduce the number of *CUP-Lists* generated from the *UP-List* for *F* from 4 to 1 in this case by using the characteristics of Definition 5.

Meanwhile, when constructing a *CUP-List* for a longer pattern from certain two *UP-Lists* or *CUP-Lists*, we can observe that, even if a combined pattern satisfies the condition of *PUFP*, its real *expSup* value may not satisfy the given *minSup* threshold. However, in order to know a real *expSup* value of the pattern, we have to construct the corresponding *CUP-List* completely. After that, if the *expSup* value of the constructed *CUP-List* is smaller than *minSup*, all of the relevant operations performed for generating the list become useless works. For this reason, in order to reduce such redundant operations and improve the mining efficiency of the proposed algorithm effectively, we propose an additional pre-pruning factor and a new technique.

Definition 6. (Pre-pruning factor (ppf)) Let U_1 and U_2 be two *UP-Lists* given for constructing a *CUP-List*, C . In addition, let *size* be the number of tuples in U_1 , k be the number of tuples processed so far in U_1 ($0 \leq k \leq size$), *Cur_expSup* be an *expSup* value of C accumulated so far, and *max* be the maximum existential probability of the values of the U_1 's tuples. Then, if it is satisfied that $minSup - Cur_expSup > (size - k) * max$, the pattern corresponding to C becomes an invalid result, and we can directly cease the subsequent tasks for constructing C .

Example 6. Fig. 8 shows how to apply the proposed *ppf* technique in the process of constructing the *CUP-List* for pattern *BA* in Fig. 6(b). In Step 1, since there is a common part between the *UP-Lists* for *B* and *A*, *TID*: 200, we store the result of multiplying the corresponding existential probability values into the *CUP-List* for *BA* as shown in the figure. After that, we consider the condition of *ppf*. Since it is not true that $2.1 - 0.81 > (4 - 1) *$

0.9, we continue to construct the current *CUP-List*. In Step 2, the product of the existential probabilities corresponding to *TID*: 500 is stored into the *CUP-List*. Since the result is true, we can cease the remaining works for constructing the list for *BA* and perform subsequent mining operations. In practice, we can see that the completely constructed *CUP-List* for *BA* eventually becomes useless because its real *expSup* value is lower than *minSup*.

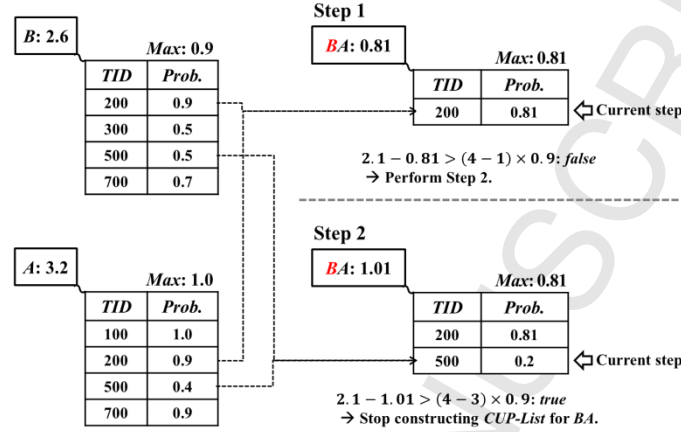


Fig. 8. Pruning of meaningless *CUP-List* using the *ppf* technique

The *ppf* technique can be utilized in not only cases of constructing *CUP-Lists* from *UP-Lists*, but also cases of generating *CUP-Lists* recursively from other *CUP-Lists*. Note that we found through various performance tests that checking the *ppf* condition for every two tuples was more efficient than doing for each tuple. Therefore, the *ppf* technique is applied in the process of *UP-List* and *CUP-List* constructions for every two tuples.

3.4.3. Speedup techniques of the proposed method without additional use of memory

In addition to the performance improving techniques proposed in the previous section, we also suggest efficient techniques that can improve the mining speed of the proposed algorithm without spending any additional memory.

Recall that additional considerations are applied when *CUP-Lists* for longer patterns are created from previous *CUP-Lists* as shown in Fig. 7. In order to update tuple information of *CUP-Lists* and compute their *expSup* values in this way, we should continue to maintain and consider the tuple information of *UP-Lists*. For example, in the case of the *CUP-List* construction for pattern *BAD*, we have to additionally consider the tuple information of the *UP-List* for *D* in order to store an exact existential probability for each tuple of the *CUP-List*. However, by using the characteristics of the proposed data structures, we can improve the previous manner of the *CUP-List* construction. In other words, we can calculate the existential probability values within the tuples of the *CUP-Lists* more efficiently in the following manner. Let C_1 and C_2 be *CUP-Lists* and $C_1.pattern = \{x_1, x_2, \dots, x_k\}$ and $C_2.pattern = \{y_1, y_2, \dots, y_k\}$ be the patterns of C_1 and C_2 , respectively, where $x_1 = y_1, x_2 = y_2, \dots, x_{k-1} = y_{k-1}$

according to Lemma 1. Then, for a *CUP-List* that can be generated from C_1 and C_2 , C' , we can easily obtain the existential probability of each tuple in C' by multiplying the existential probabilities of the corresponding tuples in C_1 and C_2 and dividing the product into the existential probability of their common part, i.e., prefix.

Lemma 3. Calculating *expSup* values of *CUP-Lists* using the characteristics of prefix guarantees exact computation results, which are equal to those of a naïve method.

Proof. Given two *CUP-Lists*, C_1 and C_2 , a *CUP-List* for a longer pattern, C' , can be generated through the combination of them. A naïve manner for calculating an existential probability for each tuple in C' is to compute $p(x_1) * p(x_2) * \dots * p(x_k) * p(y_k)$ from $C_1.pattern = \{x_1, x_2, \dots, x_k\}$ and $C_2.pattern = \{y_1, y_2, \dots, y_k\}$ ($p(x_k)$ means an existential probability corresponding to item x_k). There is no doubt that it is an exact result because such a calculation process is the same method as that of Equation (2). On the other hand, let us consider calculating an existential probability for each tuple in C' based on the newly proposed efficient method. From $C_1.pattern$ and $C_2.pattern$, their common part, i.e., prefix, becomes $\{x_1, x_2, \dots, x_{k-1}\}$. Then, we multiply the existential probability of each tuple in C_1 by that of C_2 and divide each product result into the existential probability corresponding to the current prefix. In other words, The calculation result becomes $\{ \{p(x_1) * \{p(x_2) * \dots * p(x_k)\} * \{p(y_1) * \{p(y_2) * \dots * p(y_k)\}\} / \{p(x_1) * \{p(x_2) * \dots * p(x_{k-1})\}\} = \{p(x_k)\} * \{p(y_1) * p(y_2) * \dots * p(y_k)\}\}$, where each y is equal to x except for x_k and y_k according to Lemma 1. Therefore, the final result becomes $p(x_1) * p(x_2) * \dots * p(x_k) * p(y_k)$. From the above relations, we can determine that the proposed method can draw exact *expSup* values from given *CUP-Lists*. ■

If we apply the aforementioned method, we do not need to consider any tuple information of relevant *UP-Lists* in the process of generating *CUP-Lists* recursively. Instead, we only have to know the existential probability of the prefix of previous *CUP-Lists* participating in the process of the current *CUP-List* construction, where the necessary data can easily be obtained from the previous *CUP-Lists*. Furthermore, the above work becomes simpler and easier in terms of algorithm implementation because we only have to send the existential probability value of the current prefix as a parameter for each phase of recursive calls. In addition, the deeper the depth of recursive calls for *CUP-List* constructions, the better the efficiency of the proposed method is.

Example 7. Consider applying the newly proposed method for calculating *expSup* value more efficiently in the process of creating *CUP-Lists* recursively from the *CUP-Lists* shown in Fig. 7. Note that the *CUP-Lists* for *BAD*, *BAC*, and *BDC* shown in Fig. 9 are removed in advance by the pruning techniques proposed in the previous section; therefore, they are not actually created in the mining process. However, in order to express the currently proposed technique, we assume that they are generated. As shown in Fig. 9, we can determine that the result of

each CUP-List construction is the same as that of the naïve manner. When patterns *BAD*, *BAC*, and *BDC* are combined, item *B* becomes prefix. Similarly, if *BADC* is combined from *BAD* and *BAC* (It is just an assumption. In the current mining step, the *CUP-List* for *BADC* is not actually created because of the current *minSup* setting), the corresponding prefix becomes *BA*.

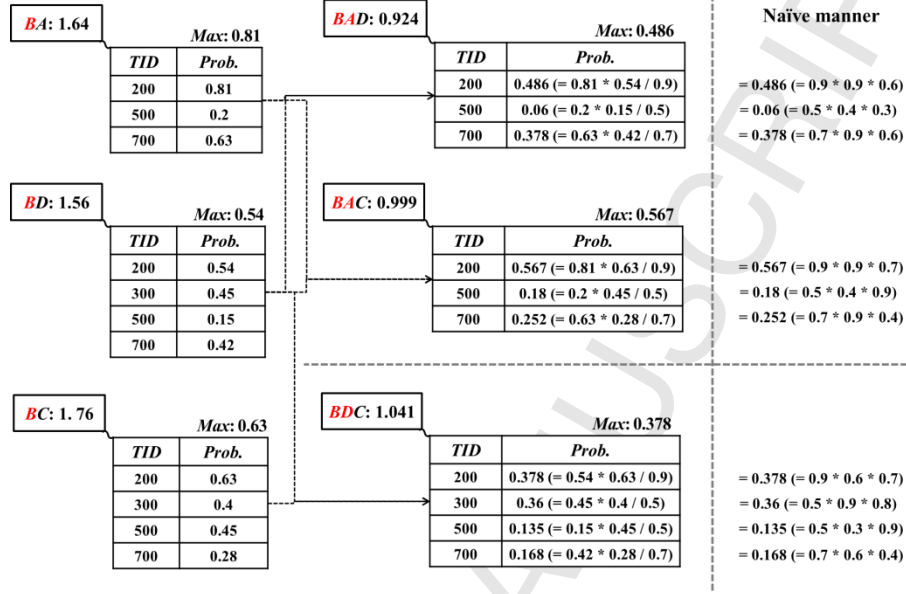


Fig. 9. Pruning of meaningless *CUP-List* using the *ppf* technique

Since the proposed technique is to improve the naïve manner more efficiently without using any extra data structure, it does not cause additional memory consumption. Although its concept is simple, it is a strong technique that can improve the algorithm performance by saving a numerous number of operations necessary for the *UFP* mining.

Another speedup technique for the proposed algorithm is a method that can more efficiently compare the tuple information of *UP-Lists* and *CUP-Lists*. Recall that the proposed algorithm recursively constructs *CUP-Lists* for longer patterns from each pair of *UP-Lists* or other *CUP-Lists* in order to mine all of the possible *UFPs*. Hence, in the process of constructing each *CUP-List*, tuple comparing operations between *UP-Lists* or *CUP-Lists* participating in the current *CUP-List* generation occur very frequently. Let U_1 and U_2 be *UP-Lists* participating in the construction of a *CUP-List* for a pattern with a longer length, and m and n be the number of tuples in U_1 and U_2 , respectively. Then, for the tuples in U_1 , the algorithm has to perform the tuple comparing operations by $m * n$ (in a naïve manner) or $m * \log_2 n$ (in a binary search manner) in order to find the same *TID* parts from the tuples in U_2 . However, by utilizing the characteristics of the proposed data structures, we can allow the algorithm to conduct the comparison works more efficiently. Recall that tuples in each of the proposed *UP-List* and *CUP-List* structures are sorted and stored in a support ascending order of their own *TIDs*. Hence, we can

apply a linear search manner into the tuple comparing processes. Let $T_{U1} = \{t_1, t_2, \dots, t_m\}$ and $T_{U2} = \{t'_1, t'_2, \dots, t'_n\}$ be tuple sets of U_1 and U_2 , respectively. Then, when t_i ($1 \leq i \leq m$) is compared to t'_j ($1 \leq j \leq n$), 1) if TID of t_i is smaller than that of t'_j , t_{i+1} is compared to t'_j again in the next step, 2) if TID of t_i is larger than that of t'_j , t_i is compared to t'_{j+1} in the next step, and 3) if they have the same TID , t_{i+1} and t'_{j+1} are compared to each other in the next step. By using this technique, we can reduce the number of comparison operations up to $m + n$.

3.5. LUNA algorithm

In this section, we describe an overall procedure of *LUNA*, the proposed exact *UFP* mining algorithm that applies the suggested data structures and mining techniques as well as various performance improving techniques. Fig. 10 shows the overall pseudo code of the proposed algorithm. *LUNA* first scans a given uncertain database, D , and computes the *expSup* value for each item composing D (lines 01-03). After that, the algorithm discards items with lower *expSup* values than a given *minSup* and calculates a support ascending order of the remaining items (lines 04-05). Moreover, these items are extracted as 1-length *UFPs* (line 06). After a series of initialization processes for *UP-Lists* are conducted (line 07), the second database scanning operations are performed to construct *UP-Lists* (lines 08-10). In this process, tuple data and *Max* information for each *UP-List* are updated according to the method mentioned in Section 3.4.1. Thereafter, the algorithm performs a series of works for constructing *CUP-Lists* from the generated *UP-Lists* (lines 11-26). Once an *UP-List* is selected (line 11), the algorithm initializes a set of *CUP-Lists*, C , and a prefix itemset, *pref* (line 12), and sets the item of the currently selected *UP-List* to *pref* (line 13). Then, another *UP-List* after the currently selected one is chosen again (line 14). After calculating an overestimated *expSup* value from the selected two *UP-Lists*, the algorithm performs subsequent tasks for constructing a *CUP-List* from them if the overestimated value is not smaller than *minSup* (lines 15-21). In the middle of generating the *CUP-List*, it can be possible to prune it according to the proposed pruning techniques. The proposed algorithm improves its mining efficiency by utilizing the existential probability information of the prefix. If *expSup* of the constructed *CUP-List* is higher than or equal to *minSup* (line 17), the *CUP-List* is stored into a set of *CUP-Lists* for the next mining task (line 18) and the pattern corresponding to the list is extracted as a valid *UFP* (line 19).

After a series of works for the *UP-List* selected first, u_k , are finished, if the number of *CUP-Lists* contained in C is higher than 1, sub-procedure *LUNA_growth* is called to mine *UFPs* with longer lengths recursively. If the algorithm conducts the above operations with respect to all the *UP-Lists*, it can provide a complete set of *UFPs* from the given uncertain database.

Procedure: LUNA

Input : an uncertain database, $D = \{T_1, T_2, \dots, T_n\}$
a given minimum support threshold, $minSup$

Output : a set of UFP results, R

Variable: a set of UP-Lists, U
a set of CUP-Lists, C
a prefix itemset, $pref$

```

01. for each  $T_i$  in  $D$  // the first database scan
02.     accumulate the existential probabilities for the items in  $T_i$  // calculate  $expSup$  for each item
03. end for
04. delete items such that their  $expSup$  values  $< minSup$  // prune invalid items
05. sort remaining items in their support ascending order // according to Lemma 2
06.  $R \leftarrow R \cup$  the sorted items // 1-length UFPs
07. set item names and  $expSup$  values of  $U$ 
08. for each  $T_i$  in  $D$  // the second database scan
09.     update the tuple and  $max$  information of each UP-List,  $u_k$ , in  $U$ 
10. end for // UP-List construction finished
11. for each  $u_k$  in  $U$ 
12.     initialize  $C$  and  $pref$ 
13.     insert  $u_k.item$  into  $pref$ 
14.     for each  $u_l$  in  $U$  //  $k < l$ 
15.         if  $((u_k.expSup * max \text{ of } u_l) \geq minSup)$  // pattern pruning based on the overestimation method
16.             generate a CUP-List,  $c$ , from  $u_k$  and  $u_l$  // pruning  $c$  is possible according to the  $ppf$  technique
17.                 // existential probability information of  $pref$  is used according to Lemma 3
18.             if  $(c.expSup \geq minSup)$ 
19.                  $C \leftarrow C \cup c$ 
20.                  $R \leftarrow R \cup c.pattern$  // 2-length UFPs
21.             end if
22.         end if
23.     end for
24.     if (# of elements in  $C > 1$ )
25.         call  $LUNA\_growth(C, pref)$  // recursive calls
26.     end if
27. end for // UFP mining operations ended
28. return  $R$ 

```

Sub_procedure: LUNA_growth(a set of CUP-Lists, C , a prefix itemset, $pref$)

```

28. for each  $c_k$  in  $C$ 
29.     initialize a set of CUP-Lists,  $C'$  // for patterns with a longer length
30.     initialize a prefix itemset for the next step,  $pref'$ , with  $pref$  and  $c_k$  // used for the next recursive phase
31.     for each  $c_l$  in  $C$  //  $k < l$ 
32.         if  $((c_k.expSup * max \text{ of } c_l) \geq minSup)$  // pattern pruning based on the overestimation method
33.             generate a CUP-List,  $c'$ , from  $c_k$  and  $c_l$  // pruning  $c'$  is possible according to the  $ppf$  technique
34.                 // existential probability information of  $pref$  is used according to Lemma 3
35.             if  $(c'.expSup \geq minSup)$ 
36.                  $C' \leftarrow C' \cup c'$ 
37.                  $R \leftarrow R \cup c'.pattern$  // UFPs with 3 or more lengths
38.             end if
39.         end if
40.     end for
41.     if (# of elements in  $C' > 1$ )
42.         call  $LUNA\_growth(C', pref')$  // recursive calls
43.     end if
44. end for

```

Fig. 10. Procedure: LUNA

The sub-procedure, $LUNA_growth$, is recursively called to construct CUP-Lists and extract UFPs with 3 or longer lengths (lines 28-43). The overall flow of this sub-procedure is similar to the part between lines 11-26 in the main procedure. The major differences between them are that 1) the prefix information is continually updated to perform the mining operations more efficiently for each call of the sub-procedure (line 30) and 2) CUP-Lists for longer patterns are recursively generated from previous CUP-Lists until there is no more CUP-List to be combined (lines 31-42).

3.6. Complexity analysis of the proposed method

In this section, we conduct theoretical analysis for the proposed method by calculating its time and space complexity. In addition to the empirical performance evaluation through various real and synthetic datasets shown in the next section, we demonstrate that the proposed algorithm outperforms previous methods in both theoretical and empirical aspects. The big- O notation is employed to compare the complexity results of the algorithms. Table 3 shows symbol definitions for algorithm complexity.

Table 3. Symbol definitions for algorithm complexity

Symbol	Meaning
n_t	The number of transactions in a given uncertain database, D
n_i	The number of items in each transaction of D
$n_{i(D)}$	The number of distinct items in D
N	The number of all items in D
P_1	A set of 1-length $UFPs$ in D such that $expSups$ of them $\geq minSup$
P_k	A set of k -length $UFPs$ in D such that $expSups$ of them $\geq minSup$
${}_kC_x$	Combinations of k elements in groups of x elements

3.6.1. Analysis of time complexity

We first analyze time complexity of the proposed algorithm. Note that the complexity of our method is compared to that of *AT-mine* [40], which has the best performance among the previous exact uncertain pattern mining approaches. Table 4 presents symbol definitions for time complexity.

Table 4. Symbols for time complexity

Symbol	Meaning
$Cost(D)$	Time cost for scanning a given uncertain database, D
$Cost(P_1)$	Time cost for sorting a set of 1-length $UFPs$, P_1
$Cost(L)$	Time cost for constructing a set of all the possible UP -Lists and CUP -Lists, L , from D
$Cost(T)$	Time cost for constructing a set of a global tree and all of the possible conditional trees, T , from D
$Time(LUNA)$	Entire time cost for mining $UFPs$ in $LUNA$ (denoted as $2 \times Cost(D) + Cost(P_1) + Cost(L)$)
$Time(AT-mine)$	Entire time cost for mining $UFPs$ in $AT-Mine$ (denoted as $2 \times Cost(D) + Cost(P_1) + Cost(T)$)

Recall that both our method and the compared one scan a given uncertain database twice in order to construct their own data structures and mine $UFPs$. The worst case scenario of the proposed algorithm assumes that each transaction composing D has the same items and a given threshold is 0 because the largest number of list combinations occurs in this case. As shown in Table 4, the entire time cost for mining $UFPs$ in $LUNA$, denoted as $Time(LUNA)$, is composed of $2 \times Cost(D) + Cost(P_1) + Cost(L)$. $Cost(D)$ of $LUNA$ is calculated as follows. Assuming that time for processing each item is 1, $Cost(D)$ becomes $O(N)$, which can also be denoted as $O(n_t \times n_i)$ since each transaction has the same items in the worst case scenario of $LUNA$. After the first database scan, $LUNA$ computes a support ascending order for valid items satisfying the given threshold condition among the items of D . Note that the number of valid items, i.e., 1-length $UFPs$, becomes $n_{i(D)}$ since the threshold is currently set to 0 and there is no pruned item in this case. We employ the quick sort technique known for

guaranteeing the best efficiency in general (its time complexity is $O(n \log_2 n)$). Then, $Cost(P_1)$ becomes $O(n_{i(D)} \log_2 n_{i(D)})$, which is also the same as $O(n_i \log_2 n_i)$ since every transaction has the same items. After that, *LUNA* scans D once again to construct *UP-Lists* for the elements of P_1 . It also spends $O(n_t \times n_i)$. Meanwhile, $Cost(L)$ can be calculated as follows. Recall that *LUNA* conducts its pattern growth operations for each element of P_1 by combining the selected element and each of the others in a recursive manner until there is no more pattern to be combined. Since there is no pruned pattern combinations in the worst case, *LUNA* generates all of the patterns possible from P_1 . Let $|P_k|$ be the number of elements in P_k in *LUNA* ($1 \leq k \leq n_{i(D)}$). Then, $|P_k|$ can be denoted as follows: $|P_1| = n_{i(D)} C_1$, $|P_2| = n_{i(D)} C_2$, ..., $|P_k| = n_{i(D)} C_k$, ..., $|P_{n_{i(D)}}| = n_{i(D)} C_{n_{i(D)}}$. Therefore, the number of pattern combinations possible from P_1 becomes $|P_1| + |P_2| + \dots + |P_k| + \dots + |P_{n_{i(D)}}| = n_{i(D)} C_1 + n_{i(D)} C_2 + \dots + n_{i(D)} C_k + \dots + n_{i(D)} C_{n_{i(D)}} = 2^{n_{i(D)}} - 1$. That is, it is equal to the number of elements in the power set of P_1 subtracting 1 (corresponding to an empty set). Then, $Cost(L)$ becomes $O(2^{n_{i(D)}} - 1)$. Consequently, $Time(LUNA) = O(2 \times (n_t \times n_i) + n_{i(D)} \log_2 n_{i(D)} + 2^{n_{i(D)}} - 1)$. n_i is equal to $n_{i(D)}$ in the worst case scenario. In addition, $n_t \leq n_{i(D)}$ because $n_{i(D)}$ is a variable that has the largest effect on the number of pattern combinations. Hence, $Time(LUNA) = O(2 \times (n_t \times n_{i(D)}) + n_{i(D)} \log_2 n_{i(D)} + 2^{n_{i(D)}} - 1)$. Since the highest order term is only considered in the big-O notation, the final result becomes $O(2^{n_{i(D)}})$.

Meanwhile, time complexity of *AT-mine* is calculated as follows. The worst case scenario of *AT-mine* assumes that transactions of D have very sparse features and a threshold is given as 0 because the efficiency of the algorithm becomes worse as its tree structure becomes sparser. As in the case of the proposed algorithm, *AT-mine* also conducts its mining operations within two database scans. In addition, the algorithm also performs item sorting works for valid ones (the difference between them is that *AT-mine* sorts the items in a support descending order; however, it does not affect the result of time complexity). Therefore, if the number of all the items in D , N , is set equally to *AT-mine* and *LUNA*, time complexity of *AT-mine* for two database scans and item sorting processes is equal to that of *LUNA*. However, since we assume that transactions of D are very sparse, $Cost(D)$ of *AT-mine* cannot be expressed equally to the case of *LUNA*. Hence, $Cost(D) = O(N)$. $Cost(T)$ of *AT-mine* is computed as follows. As shown in Table 4, T includes a global tree, G , and a set of multiple conditional trees generated from G , C_i ($1 \leq i \leq n_{i(D)} - 1$). Then, $Cost(T) = Cost(C_1) + Cost(C_2) + \dots + Cost(C_{n_{i(D)}-1})$, where the time cost for G , $Cost(G)$, is excluded from $Cost(T)$ because G has been already constructed in the previous database scanning step. $Cost(C_i)$ is calculated as follows. In order to construct a conditional tree from a global tree and process all of the works related to the generated tree, the algorithm needs two times of tree scans, item sorting operations, and recursive works for creating smaller conditional trees from the current conditional tree.

In the worst case, the number of nodes composing the global tree becomes N since any node in the tree does not merge with the others. The number of distinct items participating in the current conditional tree is $n_{i(D)} - i$. Let $Rcost(C_i)$ be time cost for conducting recursive processes for C_i and x be the number of conditional trees that can be generated from C_i . Then, $Rcost(C_i) = Cost(C_{i,1}) + Cost(C_{i,2}) + \dots + Cost(C_{i,x})$. $Cost(C_{i,m})$ ($1 \leq m \leq x$) also requires the same types of works as those of $Cost(C_i)$. The difference between them is that a smaller number of nodes are scanned and smaller items are sorted in the recursive processes. In summary, $Time(AT-mine) = O(2 \times N + n_{i(D)} \log_2 n_{i(D)} + \sum_{k=1}^{n_{i(D)}-1} (2 \times N + (n_{i(D)} - k) \log_2 (n_{i(D)} - k) + Rcost(C_k))) = O(2 \times N + n_{i(D)} \log_2 n_{i(D)} + 2 \times N \times (n_{i(D)} - 1) + \sum_{k=1}^{n_{i(D)}-1} ((n_{i(D)} - k) \log_2 (n_{i(D)} - k) + Rcost(C_k))) = O(2 \times n_{i(D)} \times N + n_{i(D)} \log_2 n_{i(D)} + \sum_{k=1}^{n_{i(D)}-1} ((n_{i(D)} - k) \log_2 (n_{i(D)} - k) + Rcost(C_k)))$. Then, the final result of the big- O notation considering the highest order term becomes $O(2 \times n_{i(D)} \times N)$. Consequently, $Time(LUNA)$ is more efficient than $Time(AT-mine)$ in a theoretical aspect.

From the analyzed results of time complexity, we can determine that *LUNA* theoretically outperforms *AT-mine*. However, it does not mean that the proposed algorithm is actually faster than the competitors because the above result is a theoretical analysis. For this reason, we also conduct empirical experiments for the algorithms with various real and synthetic datasets in the subsequent section in order to demonstrate that the proposed method is more efficient than the previous approaches in both of the aspects.

3.6.2. Analysis of space complexity

Recall that *LUNA* permanently generates and maintains *UP-Lists* on the main memory during the mining process while temporarily creating and using *CUP-Lists* if necessary. In a similar way, *AT-mine* also permanently constructs and manages a global *AT-Tree* while temporarily generating and employing conditional *AT-Trees*. The worst case scenario of these two algorithms is when each of them maximally uses memory during their own mining processes. In other words, this case occurs when recursive processes of *CUP-List* or conditional *AT-Tree* construction are maximally performed. We conduct theoretical analysis by comparing results of space complexity between the algorithms. Table 5 shows symbol definitions for space complexity.

Table 5. Symbols for space complexity

Symbol	Meaning
$Cost(U)$	Space cost for constructing and maintaining <i>UP-Lists</i>
$Cost(U')$	Space cost for recursively generating <i>CUP-Lists</i> to a maximum
$Cost(A)$	Space cost for constructing and maintaining a global <i>AT-Tree</i>
$Cost(A')$	Space cost for recursively generating conditional <i>AT-Trees</i> to a maximum
$Space(LUNA)$	Entire space cost for mining <i>UFPs</i> in <i>LUNA</i> (denoted as $Cost(U) + Cost(U')$)
$Space(AT-mine)$	Entire space cost for mining <i>UFPs</i> in <i>AT-Mine</i> (denoted as $Cost(A) + Cost(A')$)

Let us consider memory complexity of *LUNA*. Since *UP-Lists* are maintained until the mining process of *LUNA* is finished, space cost for them, $Cost(U)$, is continuously considered in $Space(LUNA)$. There is no pruned item in the worst case since the threshold is given as 0. Therefore, $n_{i(D)}$ *UP-Lists* are generated. Assume that space costs for storing a string, an integer, and a real number are 1, 1, and 2, respectively (based on the definitions of variable sizes in programming language). Then, space cost for an *UP-List*, $Cost(U_k)$ ($1 \leq k \leq n_{i(D)}$), includes creating a string for storing an item, a real number for a maximum probability value, a set of integers for storing *TID* information, and a set of real numbers for probabilities corresponding to *TIDs*. Then, in the worst case scenario, $Cost(U_k) = O(1 + 2 + n_t + 2n_t) = O(3 + 3n_t)$. $Cost(U) = Cost(U_1) + Cost(U_2) + \dots + Cost(U_{n_{i(D)}}) = O(\sum_{k=1}^{n_{i(D)}} (3 + 3n_t)) = O(3 \times n_{i(D)} + 3 \times n_t \times n_{i(D)})$. Meanwhile, $Cost(U')$ is calculated as follows. Recall that *LUNA* recursively generates *CUP-Lists* from the constructed *UP-Lists* until there is no more *CUP-List* to be made. In the case of consuming the largest memory, i.e., the worst case, the first one is selected as a prefix among the $n_{i(D)}$ *UP-Lists*, and $n_{i(D)} - 1$ *CUP-Lists* for 2-length patterns are created through the combinations of the prefix and each of the other items. After that, $n_{i(D)} - 2$ *CUP-Lists* for 3-length patterns are recursively generated. We can determine from the above relation that the maximum number of *CUP-Lists* that can recursively be generated from given *UP-Lists* becomes $(n_{i(D)} - 1) + (n_{i(D)} - 2) + \dots + 1 = \frac{n_{i(D)} \times (n_{i(D)} + 1)}{2} - n_{i(D)}$ $= \frac{n_{i(D)} \times (n_{i(D)} - 1)}{2}$. Therefore, $Cost(U') = O(\frac{n_{i(D)} \times (n_{i(D)} - 1)}{2})$. From the above results, $Space(LUNA) = O(3 \times n_{i(D)} + 3 \times n_t \times n_{i(D)} + (\frac{n_{i(D)} \times (n_{i(D)} - 1)}{2}))$, where the final result considering the highest order term becomes $Space(LUNA) = O(\frac{n_{i(D)}^2}{2})$.

The calculation process of $Space(AT-mine)$ is as follows. $Cost(A)$ is composed of the space costs for generating a header table of a global *AT-Tree*, its prefix tree, and a probability list, called *ProArr*. The header table has 3 columns, Item, expCount, and Frequency, where each of them is composed of $n_{i(D)}$ elements. The table also stores $n_{i(D)}$ node links for tree traversal. We assume that the space cost for each node link is 1. Then, space cost for the header table becomes $O(n_{i(D)} \times (1 + 2 + 1 + 1)) = O(5n_{i(D)})$. The prefix tree is composed of N nodes because there is no node sharing effect in the worst case. Among them, n_t nodes are tail nodes with additional information. Each node has its name label, parent node pointer, child node pointers, and node link, where the number of child node pointers is fixed to 1 because of the characteristics of the worst case. In addition, each tail node has variables for storing the length of the current path and *TID* information. Hence, space cost for the prefix tree becomes $O(N \times (1 + 1 + 1 + 1) + n_t \times (1 + 1)) = O(4N + 2n_t)$. *ProArr* consists of two columns, *TID* and Probabilities. *TID* has n_t elements and each element has n_i probabilities, where the total number of

probabilities stored in *ProArr* can also be denoted as N . Therefore, space cost for *ProArr* becomes $O(n_t + 2N)$. From the above results, $Cost(A) = O(5n_{i(D)} + 4N + 2n_t + n_t + 2N) = O(5n_{i(D)} + 6N + 3n_t)$. Let us consider calculating $Cost(A')$. The structures of a global *AT-Tree* and a conditional *AT-Tree* are very similar to each other. The only difference is that tails nodes of the conditional tree have two types of additional information, item indexes and base probabilities for calculating expected support values in *AT-mine*. For each tail node in the conditional tree, the number of item indexes is between 1 and n_i (denoted as n_i'), and the number of base probabilities is equal to that of *TID* indexes (in the worst case, it becomes 1). Hence, the space cost for constructing a conditional prefix tree from the global tree becomes $O((N - 1) \times (4) + n_t' \times (3 + n_i'))$. Whenever a recursive call for the conditional tree construction occurs, N decreased by 1. The space cost for generating a header table for the conditional *AT-Tree* is $O(5 \times (n_{i(D)} - 1))$. Note that *ProArr* is not considered since it is not generated in the recursive process. In the worst case scenario, recursive calls can be performed $n_{i(D)} - 1$ times. Then,

$$Cost(A') = O\left(\sum_{k=1}^{n_{i(D)}-1} (5 \times (n_{i(D)} - k) + 4 \times (N - k) + n_t' \times (3 + n_i'))\right) =$$

$$O\left(\sum_{k=1}^{n_{i(D)}-1} (5n_{i(D)} - 9k + 4N + 3n_t' + n_t' \times n_i')\right) = O((n_{i(D)} - 1) \times 5n_{i(D)} - 9 \times \frac{(n_{i(D)}-1)(n_{i(D)})}{2} +$$

$$(n_{i(D)} - 1) \times 4N + (n_{i(D)} - 1) \times (3n_t' + n_t' \times n_i')).$$

From the above results, $Space(AT-mine) = O(5n_{i(D)} + 6N + 3n_t + (n_{i(D)} - 1) \times 5n_{i(D)} - 9 \times \frac{(n_{i(D)}-1)(n_{i(D)})}{2} + (n_{i(D)} - 1) \times 4N + (n_{i(D)} - 1) \times (3n_t' + n_t' \times n_i')) = O(2N + 3n_t + \frac{1}{2}n_{i(D)}^2 - \frac{9}{2}n_{i(D)} + 4 \times n_{i(D)} \times N + (n_{i(D)} - 1) \times (3n_t' + n_t' \times n_i'))$. If we consider the highest order term of the result, $Space(AT-mine) = O(4 \times n_{i(D)} \times N)$. Then, we can determine that $Space(LUNA)$ is also more efficient than $Space(AT-mine)$ in the space complexity aspect.

The aforementioned results of complexity analysis presents that the proposed algorithm outperforms the previous state-of-the-art approach in terms of both time and space. However, since they are theoretical results, we need to conduct further tests for evaluating algorithm performance in an empirical way.

4. Performance evaluation

In this section, we compare the proposed algorithm with state-of-the-art exact *UFP* mining methods, *CUFP-mine* [22] and *AT-mine* [40], and provide extensive, comprehensive results of performance evaluation for the algorithms. Various real and synthetic datasets are used to show experimental results of the algorithms with respect to different mining environments more clearly.

4.1. Environmental settings

Recall that the proposed algorithm, *LUNA*, is an approach that can always extract results of *UFP* mining without

any false positives from given uncertain databases. Hence, among the various UFP mining methods shown in Table 1, *CUFP-mine* [22] and *AT-mine* [40] are most comparable to our method. *CUFP-mine* is a tree-based method that finds exact results of UFPs without employing any pattern growth manner based on recursive calls. *AT-mine* also mines exact UFP results as in the cases of *CUFP-mine* and the proposed algorithm. The algorithm maintains and manages its own tree structure, *AT-Tree*. One of the biggest differences from *CUFP-mine* is that *AT-mine* conducts pattern growth operations similar to those of *FP-Growth* [12]. The above algorithms always extract the same UFP results with respect to all the types of mining environments since they are all fundamental UFP mining algorithms without generating any false positive.

The compared algorithms were implemented in the C++ language and executed in a computer with 4GHz CPU, 32GB RAM, and Windows 7 OS. Tables 3 and 4 show characteristics of the datasets used for our experiments.

Table 6. Characteristics of real datasets

Dataset	Num. of Trans.	Num. of Items	Avg. Trans. Size	Data Size
Accidents	340,183	468	33.8	33.8 MB
Chain-store	1,112,949	46,086	43.0	8.82 MB
Connect	67,557	129	8.1	31.40 MB
Kosarak	990,002	41,270	23.0	0.55 MB
Mushroom	8,124	120	74.0	15.90 MB
Pumsb	49,046	2,113	10.3	3.97 MB

Table 7. Characteristics of synthetic datasets

Dataset	Num. of Trans.	Num. of Items	Avg. Trans. Size	Data Size
T10I4DxK	100,000	1,000 (fixed)	10 (fixed)	3.83 MB
	-			-
	1,000,000			38.30 MB
Tx ₁ Lx ₂ Nx ₃	100,000	10,000	10	4.81 MB
	(fixed)	-	-	-
		40,000	40	21.90 MB

Table 6 shows detailed features of the real datasets used in our runtime and memory usage tests. They are well-known benchmark datasets, which are frequently used to test algorithm performance objectively in the frequent pattern mining area. The dense datasets, Accidents and Pumsb, are composed of anonymous traffic accident data and census data for population and housing, respectively. The other dense datasets, Connect and Mushroom, include online connection data and characteristic data of various mushroom species, respectively. Kosarak is a sparse dataset composed of click-stream data from a Hungarian news portal. These five real datasets are available at the Frequent Itemset Mining (FIMI) dataset repository (<http://fimi.cs.helsinki.fi/data/>). The Chain-store dataset is also sparse and composed of purchase data by customers in a major chain in California. This dataset is available at NU-MineBench 2.0 [31]. Table 7 shows detailed features of the synthetic datasets employed in our scalability tests. They were created from the IBM dataset generator (available at <http://www.almaden.ibm.com/software/projects/hdb/resources.shtml>). As shown in the table two different

groups of data are used to test scalability performance of the algorithms. The first one, T10I4Dx (Dx: the number of transactions ($100K \leq x \leq 1,000K$)), is a synthetic dataset group featuring that the number of transactions is changed from 100k to 1,000k while the other attributes are fixed. The second one, $Tx_1Lx_2Nx_3$ (Tx_1 : an average length of transactions ($10 \leq x_1 \leq 40$); Lx_2 : the maximum number of discoverable frequent patterns ($1,000 \leq x_2 \leq 4,000$); Nx_3 : the number of distinct items ($10,000 \leq x_3 \leq 40,000$)), features that the number of attributes increases while the number of transactions is fixed.

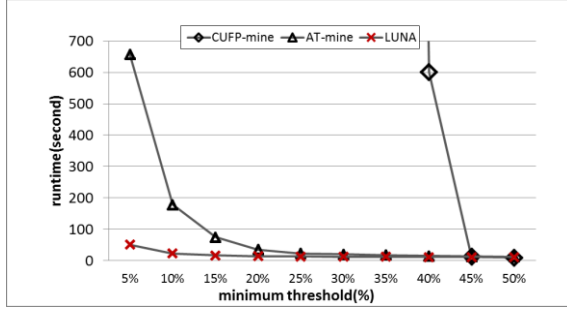


Fig. 11. Runtime results (Accidents)

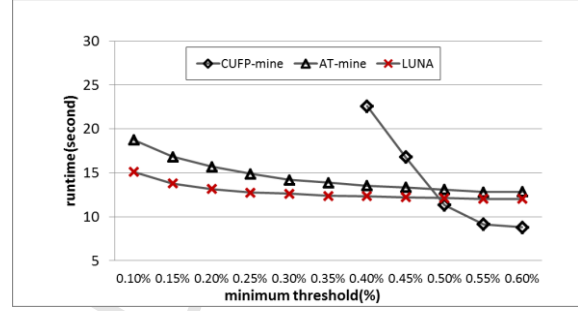


Fig. 12. Runtime results (Chain-store)

4.2. Runtime experiment

In this section, we test runtime of the compared algorithms, *LUNA*, *AT-mine*, and *CUFP-mine*, with respect to the real datasets shown in Table 6, Accidents, Chain-store, Connect, Kosarak, Mushroom, and Pumsb, and then we analyze the characteristics of them through the experimental results. Figs. 11-16 show the runtime results of the algorithms. As shown in the figures, the proposed algorithm guarantees the best runtime efficiency in almost all cases. Meanwhile, the *CUFP-mine* algorithm has the worst runtime performance in almost all cases. Especially, the algorithm fails to operate normally for the Kosarak dataset because of its memory overflow problem as shown in Fig. 14. Recall that *CUFP-mine* constructs its own tree structure, *CUFP-Tree*, to extract all of the possible *UFPs*, where each of the tree's nodes stores all of the possible pattern combinations that can be generated from the item of the node. Therefore, the size of the tree becomes exponentially larger as the size of a given dataset becomes larger and the minimum support threshold becomes lower. In Fig. 11, all the algorithms have similar runtime efficiency until the minimum support threshold is 45%. However, as the threshold becomes lower, the runtime result of *CUFP-mine* becomes much worse than those of others. *AT-mine* shows runtime performance as good as that of our algorithm until the threshold is 25%; meanwhile, the gaps between *AT-mine* and *LUNA* become more significant at lower threshold settings. Note that an algorithm is regarded as a more efficient approach in the pattern mining area when it guarantees better performance at lower threshold settings compared to previous ones. In this regard, the proposed algorithm is more efficient than the others as shown in the figure. For example, when the threshold is 5%, *LUNA* is approximately 13 times faster than *AT-mine*. Such a

tendency is also observed in the other experimental results. In Fig. 12, we can see that the performance of *CUFP-mine* is best until the threshold is 0.5%. However, the necessary runtime sharply increases as the threshold becomes lower. In particular, when the threshold is lower than 0.4%, *CUFP-mine* fails to mine *UFPs* since it causes memory overflow. For this reason, its runtime results after the threshold are excluded in the graph. The reason why the performance of *CUFP-mine* is sharply degraded is that the method has to generate and manage space for storing existential probabilities corresponding to all the possible pattern combinations for each node of its tree. Since the number of such combinations is small when the threshold is set relatively high, *CUFP-mine* may have good performance in a few cases as shown in Fig. 12. However, in most cases, its performance is the worst among the compared ones. Recall that *CUFP-mine* constructs a global tree and stores all the information necessary for extracting valid patterns into the tree, instead of using recursive pattern-growth manners like *AT-mine* and our method. Hence, the merit of *CUFP-mine* can occur when the size of the constructed tree is small and it is not complicated. In this case, *CUFP-mine* can efficiently extract valid pattern results by simply arranging data stored in the tree. Meanwhile, *AT-mine* and our method spend execution time recursively constructing their own data structures and performing their own mining operations regardless of the characteristics of given data and threshold settings. For this reason, *CUFP-mine* can be more efficient in some cases as shown in Fig. 12. However, the threshold range that the algorithm can operate effectively is relatively narrow compared to those of *AT-mine* and *LUNA*. For a given dataset, situations advantageous to *CUFP-mine* are more likely to occur when the threshold is set higher because data structure with a smaller volume can be generated in these cases. However, as the threshold becomes higher, the number of generated patterns sharply decreases and no patterns are mined in the end. In Fig. 13, *CUFP-mine* also fails to operate normally when the threshold is lower than 50% for the Connect dataset. Such a tendency is also shown in the result of Fig. 16. In the case of Fig. 15, runtime performance of the algorithm is similar to that of *AT-mine* and our *LUNA*. However, its runtime result becomes exponentially larger as the threshold becomes lower as shown in the figure. Especially when the threshold is 5% in Figs. 13 and 16, the proposed algorithm is approximately 20 and 22 times faster than *AT-mine*, respectively. From the above results, we can determine that *CUFP-mine* is worst among the compared methods in almost all cases although its performance is not bad at relatively higher threshold settings. The reason for this is that *AT-tree* stores a smaller number of pattern combinations for each node and the algorithm directly extracts *UFPs* from the tree without complicated pattern growth operations in this case. Runtime performance of *AT-mine* is as good as that of *LUNA* when the threshold settings are relatively high as shown in the figures. However, its performance also becomes worse as the threshold becomes lower. On

the other hand, the proposed algorithm guarantees the best runtime performance regardless of threshold settings and dataset types. It is obvious that necessary runtime of each algorithm is increased as the threshold becomes smaller because they have to generate a larger number of *UFPs*. Nevertheless, we can see that the increasing runtime rate of the proposed algorithm is smallest among them because of the newly proposed concept for *UFP* mining, data structures, mining techniques, pre-pruning techniques, and performance improving strategies.

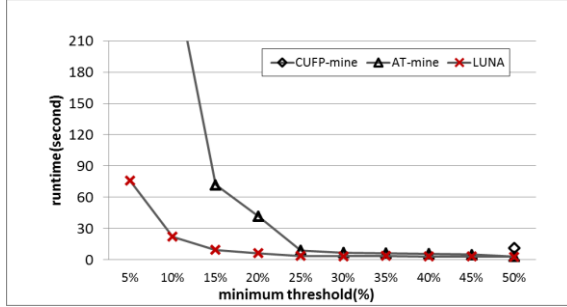


Fig. 13. Runtime results (Connect)

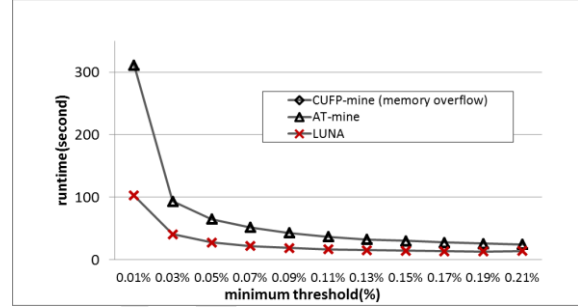


Fig. 14. Runtime results (Kosarak)

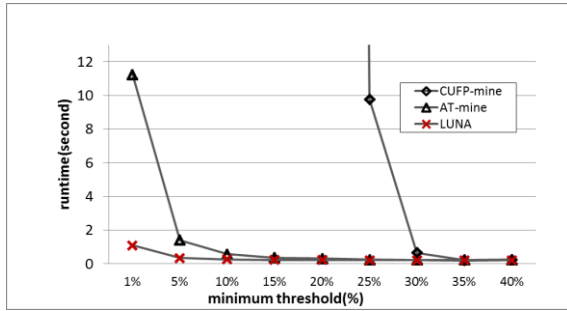


Fig. 15. Runtime results (Mushroom)

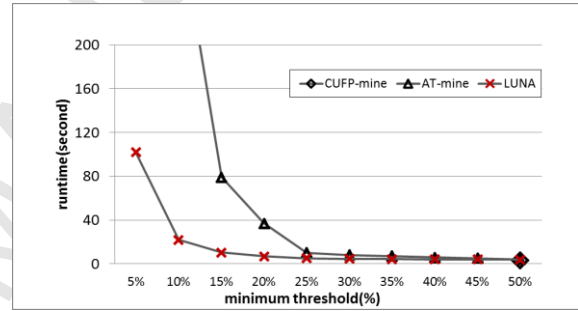


Fig. 16. Runtime results (Pumsb)

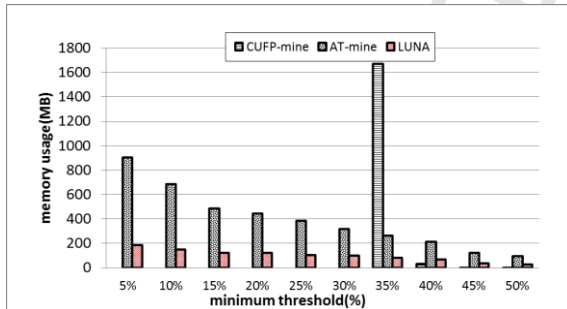


Fig. 17. Memory usage results (Accidents)

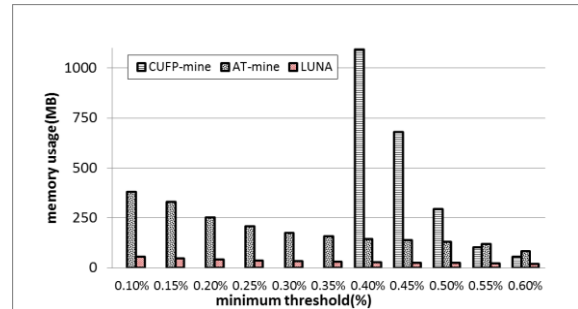


Fig. 18. Memory usage results (Chain-store)

4.3. Memory usage experiment

In this section, memory usage tests are performed for the real datasets in Table 6 as in the case of runtime experiments, where the parameter settings for the tests are the same as those of the previous runtime experiments. Fig. 17 shows the results of memory usage for the Accidents dataset. From the graph, we can observe that the proposed algorithm is most efficient among the compared ones. For example, when the threshold is 35%, memory usage of *CUFP-mine* is approximately 20 times larger than that of *LUNA*. In addition,

as the threshold becomes lower, *CUFP-mine* causes memory overflow. When the threshold is 5%, our algorithm is approximately 4.8 times more efficient than *AT-mine* as shown in the figure. Such a tendency is also similarly shown in Fig. 18, where the memory usage results of *CUFP-mine* is also excluded when the threshold settings are smaller than 0.4% because of its memory overflow problem. In Fig. 19, we can see that the memory efficiency of *CUFP-mine* is best when the threshold is 50%. However, its performance becomes even worse as the threshold becomes lower than 50%. The algorithm causes memory overflow when the threshold is 45% or less. Meanwhile, the proposed algorithm guarantees the most efficient memory usage for the given dataset and threshold settings. In Fig. 20, our algorithm also shows the most efficient and stable memory performance among them. *CUFP-mine* suffers from memory overflow with respect to all of the threshold settings. Although *AT-mine* has better memory efficiency than that of *CUFP-mine*, its performance is approximately 6 times worse than that of *LUNA* in average. In the case of Fig. 21, *CUFP-mine* has good memory efficiency until the threshold is 30%. However, as the threshold becomes lower than 30%, it requires much more memory to mine *UFPs*. Especially when the threshold is 20%, it consumes memory space approximately 128 times larger than that of *AT-mine* and 201 times larger than that of *LUNA*, respectively. The results of Fig. 22 are similar to those of Fig. 19. Obviously, the proposed algorithm also shows the best performance in this case. From the above results of the memory usage tests, we can determine that *CUFP-mine* frequently suffers from the memory overflow problem because its own tree structure requires excessive memory resources to generate and maintain numerous array structures for storing all of the possible pattern combinations. For this reason, the algorithm has the worst memory performance in almost all cases although it shows good memory efficiency in a few cases. Overall, *AT-mine* shows acceptable memory performance compared to *CUFP-mine*. However, its performance falls behind our approach. On the other hand, *LUNA* employs the newly proposed *UFP* mining concept, novel data structures and mining techniques based on the structures, and various performance improving techniques. Moreover, we can observe that the experimental results support the effectiveness of them. That is, the results demonstrate that the proposed *LUNA* algorithm can conduct *UFP* mining operations much more efficiently than the competitors as well as always mine the same pattern results as those of the compared ones. The common observation in the memory experiments is that *CUFP-mine* clearly shows its fatal memory problem as the threshold becomes lower. As shown in Fig. 2(a), each node of *CUFP-Tree* except for the child nodes of its root has its own array structure to store existential probabilities of all the possible pattern combinations. That is, if the length of a path in the tree is n , the last node of the path has to create $2^{(n-1)} - 1$ array elements, not to mention that the other nodes above the last one also have to generate an enormous number of elements. As the

size of a given dataset becomes larger and a given threshold is set lower, n can sharply increase. Then, the number of array elements is exponentially raised. That is why *CUIFP-mine* has the worst memory efficiency in most cases and even fails to operate normally in many cases. Meanwhile, *AT-mine* constructs a global *AT-Tree* and multiple conditional *AT-Trees* to mine *UFPs*. In addition, the method also generates and maintains a list of existential probabilities, *ProArr*, used to calculate expected supports of patterns. As shown in Fig. 2(b), its tree structure is composed of complicated components such as a header table, two types of nodes (ordinary and tail nodes), several types of node links, etc., in contrast to the proposed minimum list structure. For this reason, *AT-mine* requires more runtime and memory resources than those of *LUNA* in every case.

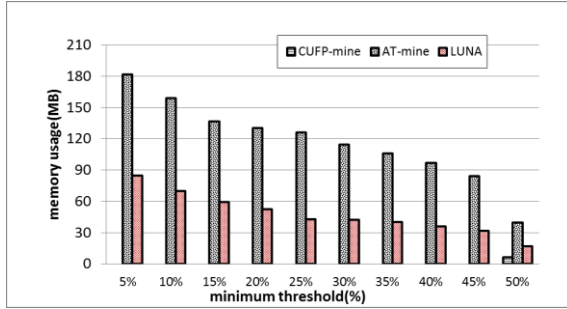


Fig. 19. Memory usage results (Connect)

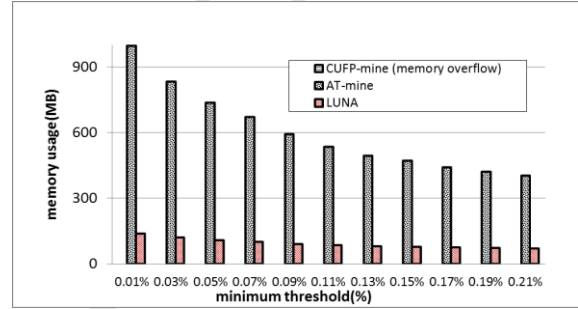


Fig. 20. Memory usage results (Kosarak)

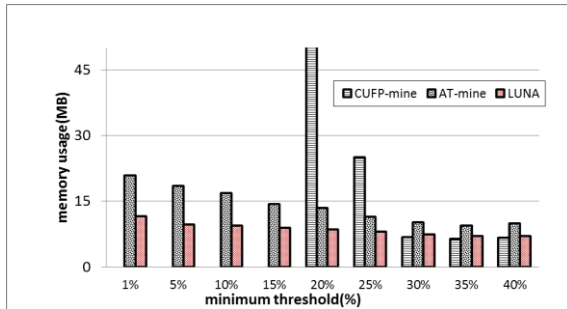


Fig. 21. Memory usage results (Mushroom)

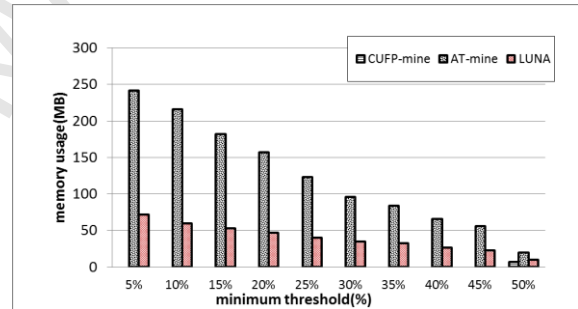


Fig. 22. Memory usage results (Pumsb)

4.4. Scalability experiment

Figs. 23 and 24 show the results of scalability tests for the synthetic datasets in Table 7, where the threshold settings are 1.2% in the first dataset group and 0.9% in the second one. The reason why we set the threshold to the above values is that *CUIFP-mine* fails to operate normally at lower threshold settings than 1.2% in the first group and 0.9% in the second one. By setting the thresholds as these values, we can compare the scalability performance of all the algorithms. Recall that the first dataset group features that the number of transactions increases from 100K to 1,000K while the number of the other attributes is fixed. Fig. 23(a) shows the runtime scalability of each algorithm for the first group. As shown in the figure, *CUIFP-mine* has the worst runtime scalability, while the other algorithms are more scalable on the dataset group. In addition, we can observe that

the proposed algorithm also guarantees the best performance. Furthermore, if the current threshold is set lower, the scalability of *LUNA* becomes better than the others because of its own data structures and various mining techniques. The algorithm also shows the best memory scalability as shown in Fig. 23(b). Since *CUFP-mine* has to construct and maintain an array structure with all the possible pattern combinations for each node of its own tree, *CUFP-Tree*, it consumes much more memory compared to the others. *AT-mine* shows better results than those of *CUFP-mine*, but its scalability is worse than that of ours because the algorithm has to maintain and manage a global *AT-Tree*, numerous conditional *AT-Trees*, and a probability list, *ProArr*. Meanwhile, the proposed algorithm does not need to construct and manage any tree structure to mine *UFPs*. All the algorithm has to mine *UFPs* is only construct the proposed minimum data structures and combine them. Hence, it can guarantee the best memory scalability as well as the best memory efficiency. The proposed algorithm also shows the best runtime and memory scalability performance for the second dataset group as shown in Fig. 24(a) and (b). Recall that the second dataset group features that the number of attributes except for the transaction number is gradually increased while the number of transactions is fixed to 100K. *CUFP-mine* shows runtime scalability as good as those of the others until the third dataset, T30L3000N30000. However, as the number of attributes becomes larger, its scalability becomes sharply worse while the runtime results of the others are stably increased. That is, *LUNA* and *AT-mine* are more scalable than *CUFP-mine* in terms of runtime. In the memory scalability test for the second dataset group, we can observe that the scalability performance of our method is better than the others. As the number of attributes becomes larger, all the algorithms require more memory resources to mine *UFPs*. However, the increasing degree of the proposed method is smallest and most stable, while that of *CUFP-mine* is largest and most unstable. Recall that *CUFP-mine* is an approach that mines uncertain frequent patterns without any recursive pattern-growth manner. Instead, it stores all of the necessary data into its own tree structure. In general, such an approach has the worst performance among the compared algorithms as shown in the figures provided in the performance evaluation section. However, in some situations, *CUFP-mine* may present better performance. Such a result can be obtained if all of the following conditions are satisfied: 1) the size of the constructed *CUFP-Tree* is small, 2) most of the paths in the tree are short, 3) the number of paths is small, and 4) many items are pruned from a given dataset. Cases satisfying all the conditions rarely occur. Of course, if we set the threshold very high, we can satisfy the above conditions. However, since the very small number of patterns or even no result can be mined in this case, it is hard to analyze given uncertain data. Moreover, the most important thing in the memory scalability test is observing how stable memory usage the compared algorithms have. In this regard, the memory scalability of *CUFP-mine* is the worst although it shows

good performance for 1000 attributes in Fig. 24(b).

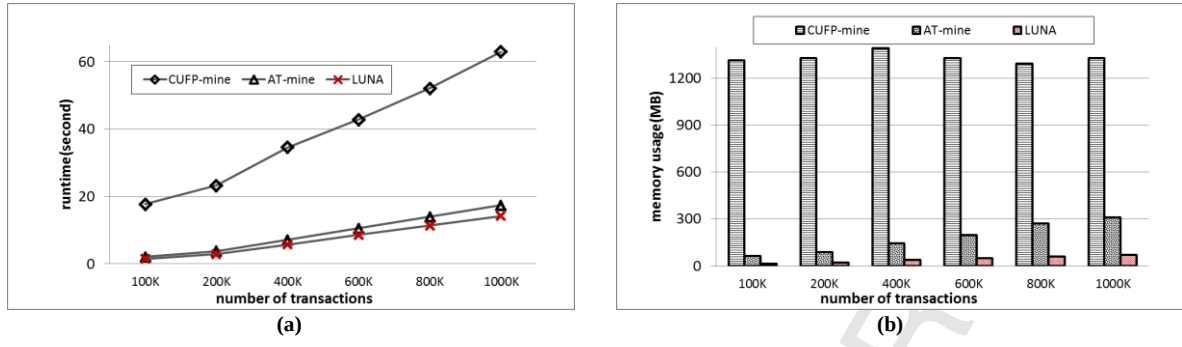


Fig. 23. Results of runtime and memory scalability (T10I4DxK)

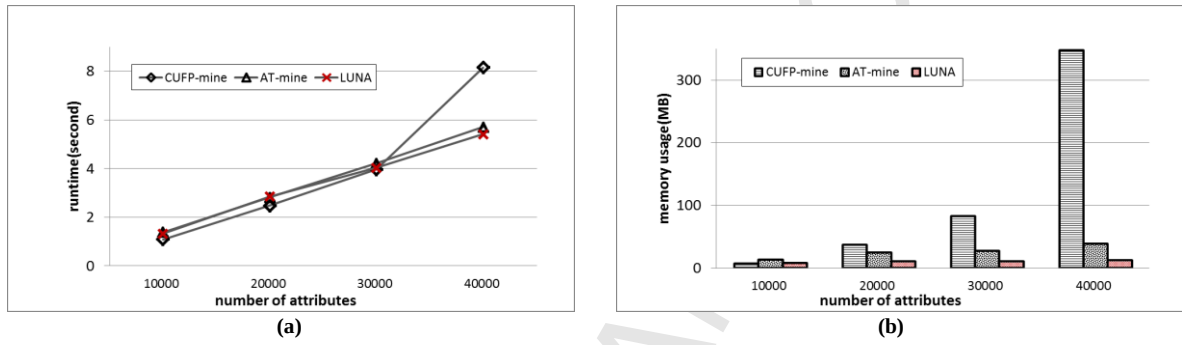


Fig. 24. Results of runtime and memory scalability ($T_{x_1}L_{x_2}N_{x_3}$)

From the above extensive results of performance evaluation for the proposed algorithm and the state-of-the-art methods, we can determine that the proposed *LUNA* algorithm outperforms the others on various *UFP* mining environments in terms of runtime, memory usage, and scalability.

5. Discussion

As shown in the above sections, the main goals of this paper have been achieved by the proposed algorithm. Results of time and space complexity, various empirical results of performance evaluation, and their analytical results proved that improvement of algorithm performance, one of the main goals, has been achieved in both theoretical and empirical aspects. Meanwhile, the second goal in this paper is to mine exact uncertain frequent pattern mining results without any false positives as well as false negatives, and this issue has also been accomplished. Firstly, let us consider why our approach can mine no false negatives. In the uncertain frequent pattern mining area, false negatives mean patterns that are actually uncertain frequent ones but misjudged as uncertain infrequent patterns. Such a phenomenon can occur when expected support values of patterns are calculated lower than their actual values in an approximation manner. Among the approximate uncertain frequent pattern mining approaches, methods based on statistical techniques such as *MBP* [41] and *IMBP* [38] can cause this false negative problem. On the other hand, the other algorithms do not mine any false negatives.

Exact approaches including ours do not estimate or distort expected support values of patterns in their own mining processes. That is, since they calculate exact expected supports without any approximation manner, they contain no false negatives. In the cases of the other approximate algorithms except for statistical technique-based ones, no false negatives are also mined because they do not estimate expected supports of patterns lower than their actual values. Secondly, let us consider why the proposed method can mine no false positives. In contrast to false negatives, false positives are patterns that are actually uncertain infrequent ones but misjudged as valid results. All the approximate methods in uncertain frequent pattern mining cause such a false positive problem since their own mining techniques can allow expected supports of patterns to be estimated higher than their actual values. On the other hand, since exact methods including ours always compute the values accurately, we do not need to consider such side effects.

Although the proposed algorithm has guaranteed the best performance in almost all cases as shown in the performance evaluation section, it may not be effective on the following situation. Recall that our method constructs an *UP-List* for each valid item of a given database. Then, they are recursively combined to extract uncertain frequent patterns. Therefore, if the number of generated *UP-Lists* is extremely large, performance degradation may occur although we apply the newly proposed strong pruning techniques in the mining process. Such a case is more likely to arise when a given database is very sparse and large and a user-specific threshold is very low. Fortunately, it is hard to satisfy all of these conditions in real world applications, but it still remains a limitation of the proposed method. To solve the issue, we are planning to devise another novel data structure and techniques in our future work. The other problem of our approach is how to find appropriate threshold settings. There are various types of data obtained in different fields and they have their own characteristics. Hence, it is hard to find proper threshold settings satisfying user requirements. If the threshold is set too low or high, a too small or large number of pattern results are mined. Moreover, proper settings can be different for each dataset. We can solve such a problem by applying the concept of top-k pattern mining into our method. This allows the algorithm to find appropriate mining results without setting any threshold value. We are also scheduled to develop this solution in our future work.

Exact approaches can be utilized effectively in many cases, but they may have limitations in a few specific situations. For example, if massive amounts of data have to be processed in real time, it is hard to provide users with a complete set of uncertain frequent patterns. In this case, approximate approaches can effectively be used since they can deal with such massive data faster although we have to endure false positive and negative problems. Moreover, since it has been found that the performance of the proposed method is better than

previous state-of-the-art methods, we can obtain an efficient approximate method by integrating the proposed techniques and the concept of approximate uncertain frequent pattern mining.

6. Conclusions

In this paper, we proposed a new efficient method for mining uncertain frequent patterns exactly without any false positive patterns. Through the newly proposed data structures, we could effectively store given uncertain data as a minimum expression and perform uncertain frequent pattern mining operations with less runtime and memory resources. In addition, a variety of the proposed performance improving techniques allowed the algorithm to conduct the mining operations more efficiently. We demonstrated the correctness of the proposed algorithm by comparing ours with previous state-of-the-art approaches that can mine exact results of uncertain frequent pattern mining. In addition, the results of performance analysis provided in the performance evaluation section showed that the proposed algorithm outperformed the competitors in various aspects such as runtime, memory usage, and scalability. The data structures and mining techniques devised in this paper can also be applied in various pattern mining areas such as dynamic pattern mining on data streams and representative pattern mining because they are techniques for fundamental uncertain frequent pattern mining that can be utilized widely as basic frameworks. Such expanded studies are scheduled to be conducted in our future work.

Acknowledgements

This research was supported by the National Research Foundation of Korea (NRF) funded by the Ministry of Education, Science and Technology (NRF No. 20152062051, and NRF No. 20155054624), and the Business for Academic-industrial Cooperative establishments funded Korea Small and Medium Business Administration in 2015 (Grants No. C0261068).

References

- [1] C.C. Aggarwal, Y. Li, J. Wang, and J. Wang, "Frequent pattern mining with uncertain data", 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp.29-37, 2009.
- [2] R. Agrawal and R. Srikant, "Fast Algorithms for Mining Association Rules", 20th International Conference on Very Large Data Bases, pp. 487-499, 1994.
- [3] I. Batal, G.F. Cooper, D. Fradkin, J.H. Harrison Jr., F. Moerchen, and M. Hauskrecht, "An efficient pattern mining approach for event detection in multivariate temporal data", Knowledge and Information Systems, vol. 46, no. 1, pp. 115-150, 2016.
- [4] J. Cai, X. Zhao, and Y. Xun, "Association rule mining method based on weighted frequent pattern tree in mobile computing environment", International Journal of Wireless and Mobile Computing, vol. 6, no. 2, pp. 193-199, 2013.
- [5] J. Chen and P. Chen, "Sequential Pattern Mining for Uncertain Data Streams using Sequential Sketch", Journal of Networks, vol. 9, no. 2, pp. 252-258, 2014.
- [6] R. Cheng, D.V. Kalashnikov, and S. Prabhakar, "Querying Imprecise Data in Moving Object Environments", IEEE Transactions on Knowledge and Data Engineering, vol. 16, no. 9, pp. 1112-1127, 2004.
- [7] C. Chui, B. Kao, and E. Hung, "Mining Frequent Itemsets from Uncertain Data", 11th Pacific-Asia Conference on Advances in Knowledge Discovery and Data Mining, pp. 47-58, 2007.
- [8] A. Cuzzocrea, C. Leung, R. MacKinnon, "Mining constrained frequent itemsets from distributed

- uncertain data”, *Future Generation Computer Systems*, vol. 37, pp. 117-126, 2014.
- [9] G. Fang, Z. Deng, and H. Ma, “Network Traffic Monitoring Based on Mining Frequent Patterns”, *Fuzzy Systems and Knowledge Discovery*, vol. 7, pp. 571-575, 2009.
- [10] G. Gonzalez, T. Tahsin, B.C. Goodale, A.C. Greene, and C.S. Greene, “Recent Advances and Emerging Applications in Text and Data Mining for Biomedical Discovery”, *Briefings in Bioinformatics*, vol. 17, no. 1, pp. 33-42, 2016.
- [11] G. Grahne and J.Zhu, “Fast Algorithms for Frequent Itemset Mining Using FP-Trees”, *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 10, pp. 1347-1362, 2005.
- [12] J. Han, J. Pei, Y. Yin and R. Mao, “Mining frequent patterns without candidate generation: a frequent-pattern tree approach”, *Data Mining and Knowledge Discovery*, vol. 8, no. 1, pp.53–87, 2004.
- [13] R. Jiang, R. Lu, and K. Choo, “Achieving high performance and privacy-preserving query over encrypted multidimensional big metering data”, *Future Generation Computer Systems*, In Press, 2016.
- [14] G. Lee, U. Yun, and H. Ryang, “Mining Weighted Erasable Patterns by using Underestimated Constraint-based Pruning Technique”, *Journal of Intelligent and Fuzzy Systems*, vol. 28, no. 3, pp. 1145-1157, 2015.
- [15] G. Lee, U. Yun, H. Ryang, and D Kim, “Multiple Minimum Support-Based Rare Graph Pattern Mining Considering Symmetry Feature-Based Growth Technique and the Differing Importance of Graph Elements”, *Symmetry*, vol. 7, no. 3, pp. 1151-1163, 2015.
- [16] G. Lee, U. Yun, and H. Ryang, “An uncertainty-based approach: Frequent itemset mining from uncertain data with different item importance”, *Knowledge-Based Systems*, vol. 90, pp. 239-256, 2015.
- [17] C.K. Leung, R.K. MacKinnon, and S.K. Tanbeer, “Tightening Upper Bounds to the Expected Support for Uncertain Frequent Pattern Mining”, *The 18th Annual Conference on Knowledge-Based and Intelligent Information & Engineering Systems*, pp. 328-337, 2014.
- [18] C.K. Leung and S.K. Tanbeer, “PUF-Tree: A Compact Tree Structure for Frequent Pattern Mining of Uncertain Data”, *The 17th Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp. 13-25, 2013.
- [19] C.K. Leung, M.A.F. Mateo, and D.A. Brajczuk, “A tree-based approach for frequent pattern mining from uncertain data”, *12th Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp.653-661, 2008.
- [20] C.K. Leung, C.L. Carmichael, and B. Hao, “Efficient mining of frequent patterns from uncertain data”, *International Conference on Data Mining Workshops*, pp.489-494, 2007.
- [21] L. Li, R. Lu, K. Choo, A. Datta, and J. Shao, “Privacy-Preserving-Outsourced Association Rule Mining on Vertically Partitioned Databases”, *IEEE Trans. Information Forensics and Security*, vol. 11, no. 8, pp. 1847-1861, 2016.
- [22] C. Lin, and T. Hong, “A new mining approach for uncertain databases using CUFP trees”, *Expert Systems with Applications*, vol. 39, no. 4, pp. 4084-4093, 2012.
- [23] X. Liu, R. Deng, K. Choo, and J. Weng, “An Efficient Privacy-Preserving Outsourced Calculation Toolkits with Multiple Keys”, *IEEE Transactions on Information Forensics and Security*, vol. PP, no. , pp. 1-14, 2016.
- [24] X. Liu, K. Choo, R. Deng, R. Lu, and J. Weng, “Efficient and Privacy-Preserving Outsourced Calculation of Rational Numbers”, *IEEE Transactions on Dependable and Secure Computing*, vol. PP, no. 99, pp. 1-14, 2016.
- [25] Y. Liu, “Mining maximal frequent patterns from univariate uncertain data”, *Intelligent Data Analysis*, vol. 18, no. 4, pp. 653-676, 2014.
- [26] C. Maione, E. Paula, M. Gallimberti, B. Batista, A. Campiglia, F. Barbosa Jr, and R.M. Barbosa, “Comparative study of data mining techniques for the authentication of organic grape juice based on ICP-MS analysis”, *Expert Systems With Applications*, vol. 49, pp. 60-73, 2016.
- [27] B. Martini and K. Choo, “Cloud storage forensics: ownCloud as a case study”, *Digital Investigation*, vol. 10, no. 4, pp. 287-299, 2013.
- [28] B. Martini and K. Choo, “An integrated conceptual digital forensic framework for cloud computing”, *Digital Investigation*, vol. 9, no. 2, pp. 71-80, 2012.
- [29] O. Osanaiye, K. Choo, and M. Dlodlo, “Distributed denial of service (DDoS) resilience in cloud: Review and conceptual cloud DDoS mitigation framework”, *Journal of Network and Computer Applications*, vol. 67, pp. 147-165, 2016.
- [30] J. Pei, J. Han, H. Lu, S. Nishio, S. Tang, and D. Yang, “H-Mine: Fast and space-preserving frequent pattern mining in a large databases”, *IIE Transactions (Institute of Industrial Engineers)*, vol.39, no.6, pp.593-605, 2007.
- [31] J. Pisharath, Y. Liu, B. Ozisikyilmaz, R. Narayanan, W. K. Liao, A. Choudhary, and G. Memik, “NU-MineBench version 2.0 dataset and technical report,” <http://cucis.ece.northwestern.edu/projects/DMS/MineBench.html>.
- [32] G. Pyun, U. Yun, and K.H. Ryu, “Efficient frequent pattern mining based on Linear Prefix Tree”, *Knowledge-Based Systems*, vol. 55, pp. 125-139, 2014.

- [33] D. Quick, K. Choo, "Forensic collection of cloud storage data: Does the act of collection result in changes to the data or its metadata?", *Digital Investigation*, vol. 10, no. 3, pp. 266-277, 2013.
- [34] N. Rahman and K. Choo, "A survey of information security incident handling in the cloud", *Computers & Security*, vol. 49, no. 45-69, 2015.
- [35] H. Ryang and U. Yun, "Fast algorithm for high utility pattern mining with the sum of item quantities", *Intelligent Data Analysis*, vol. 20, no. 2, pp. 395-415, 2016.
- [36] A. Sallaberry, N. Pecheur, S. Bringay, M. roche, and M. Teisseire, "Sequential patterns mining and gene sequence visualization to discover novelty from microarray data", *Journal of Biomedical Informatics*, vol.44, pp. 760-774, 2011.
- [37] M.Y. Su, G.J. Yu, and C.Y. Lin, "A real-time network intrusion detection system for large-scale attacks based on an incremental mining approach", *Computers & Security*, vol. 28, no. 5, pp. 301-309, 2009.
- [38] X. Sun, L. Lim and S. Wang, "An approximation algorithm of mining frequent itemsets from uncertain dataset", *International Journal of Advancements in Computing Technology*, vol.4, no.3, pp.42-49, 2012.
- [39] L. Wang, L. Feng, and M. Wu, "UDS-FIM: An Efficient Algorithm of Frequent Itemsets Mining over Uncertain Transaction Data Streams", *Journal of Software*, vol. 9, no. 1, pp. 44-56, 2014.
- [40] L. Wang, L. Feng, and M. Wu, "AT-Mine: An Efficient Algorithm of Frequent Itemset Mining on Uncertain Dataset", *Journal of Computers*, vol. 8, no. 6, pp. 1417-1426, 2013.
- [41] L. Wang, D.W. Cheung, R. Cheng, S. Lee, and X. Yang, "Efficient Mining of Frequent Itemsets on Large Uncertain Databases", *IEEE Transactions on Knowledge and Data Engineering*, vol. 24, no. 12, pp. 2170-2183, 2012.
- [42] J. Wu, F. Wang, and P. Xiang, "Automatic network clustering via density-constrained optimization with grouping operator", *Applied Soft Computing*, vol. 38, pp. 606-616, 2016.
- [43] K. Xu, K. Zou, Y. Huang, X. Yu, and X. Zhang, "Mining community and inferring friendship in mobile social networks", *Neurocomputing*, vol. 174, pp. 605-616, 2016.
- [44] M. Yiu, N. Mamoulis, X. Dai, Y. Tao, and M. Vaitis, "Efficient Evaluation of Probabilistic Advanced Spatial Queries on Existentially Uncertain Data", *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 1, pp. 108-122, 2009.
- [45] U. Yun and G. Lee, "Incremental mining of weighted maximal frequent itemsets from dynamic databases", *Expert Systems with Applications*, vol. 54, pp. 304-327, 2016.
- [46] U. Yun and G. Lee, "Sliding window based weighted erasable stream pattern mining for stream data applications", *Future Generation Computer Systems*, vol. 59, pp. 1-20, 2016.
- [47] U. Yun, D. Kim, H. Ryang, G. Lee, and K. Lee, "Mining recent high average utility patterns based on sliding window from stream data", *Journal of Intelligent and Fuzzy Systems*, vol. 30, no. 6, pp. 3605-3617, 2016.
- [48] U. Yun, G. Lee, and C. Kim, "The Smallest Valid Extension-Based Efficient, Rare Graph Pattern Mining, Considering Length-Decreasing Support Constraints and Symmetry Characteristics of Graphs", *Symmetry*, vol. 8, no. 5, 2016.
- [49] U. Yun, G. Pyun, and E. Yoon, "Efficient Mining of Robust Closed Weighted Sequential Patterns Without Information Loss", *International Journal on Artificial Intelligence Tools*, vol. 24, no. 1, 2015.
- [50] U. Yun and J. Kim, "A fast perturbation algorithm using tree structure for privacy preserving utility mining", *Expert Systems with Applications*, vol. 42, no. 3, pp. 1149-1165, 2015.
- [51] U. Yun and H. Ryang, "Incremental High Utility Pattern Mining with Static and Dynamic Databases", *Applied Intelligence*, vol. 42, no. 2, pp. 323-352, 2015.
- [52] Y. Zhang, R. Cheng, and J. Chen, "Evaluating Continuous Probabilistic Queries Over Imprecise Sensor Data", *15th International Conference on Database Systems for Advanced Applications*, pp. 535-549, 2010.
- [53] W. Zhao, R. Chellappa, P.J. Phillips, and A. Rosenfeld, "Face recognition: A literature survey", *ACM Computing Surveys*, vol. 35, pp. 399-458, 2003.
- [54] Z. Zhao, D. Yan, W. Ng, "Mining Probabilistically Frequent Sequential Patterns in Large Uncertain Databases", *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 5, pp. 1171-1184, 2014.

Biography

Name: Unil Yun

Position: Professor,

Department of Computer Engineering,

Sejong University, Seoul, Republic of Korea

Email: yunei@sejong.ac.kr

Homepage: ce.sejong.ac.kr/~yunei

Address:

143-747, Department of Computer Engineering,

Sejong University,

Seoul, Republic of Korea

Education:

Ph.D. Computer Science, Texas A&M University, December 2005

M.S. Computer Science, Korea University, February 1997

B.S. Computer Engineering, HongIk University, February 1995

Work Experience :

September 2013 – Present: Associate professor, Department of Computer Engineering, in Sejong University

September 2012 – August 2013: Associate professor, Department of Software, division of Computer Engineering, in Chungbuk National University

September 2007 – August 2012: Assistant professor at division of Computer Engineering, in Chungbuk National University

December 2006 – August 2007: Senior Member of engineering Staff, Telematics research team, Electronics and Telecommunications Research Institute

December 2005 – November 2006: Post doctoral associate at Department of Computer Science in Texas A & M University

February 1997 – June 2002: Member of Technical Staff, Research and Development Group, Multimedia Laboratory, Korea Telecom, 1997–2002

Research area :

- Data Mining (Discovering frequent patterns, association rule mining, Mining Frequent patterns with Constraints, Mining Frequent Closed patterns, Mining Frequent Sequential patterns, Web Mining, Clustering, and Classification)
- Database systems, Bioinformatics, Data Management, Large data indexing

- Information Retrieval
- Knowledge Based systems, Information Systems,
- Algorithm, computational science, performance evaluation

Publication, Patent, Awards, Journal Activities:

Please access homepage of our Lab.: ce.sejong.ac.kr/~yunei

*Biographies (Photograph)

[Click here to download high resolution image](#)

